
Análisis Estadístico de Datos de Ventas: Inferencia y Modelado



Integrantes del Grupo:
Martin Cerda Fernandez
Maximiliano Campos Camimil

Docente: Sebastián Salazar Molina
Fecha de entrega: 10 de julio de 2026

1. Introducción.....	4
1.1. Contexto del problema (Cruz Morada).....	4
1.2. Objetivos del análisis.....	4
2. Descripción de la Solución Implementada.....	5
2.1. Arquitectura del pipeline (incluye carga por chunks).....	5
2.2. Reproducibilidad (semilla CPYD_SEED).....	7
2.3. Justificación de librerías utilizadas.....	9
3. Preprocesamiento y Limpieza de Datos.....	11
3.1. Tratamiento de valores faltantes (test MCAR proxy).....	11
3.2. Detección de outliers (IQR).....	13
3.3. Variables derivadas.....	15
3.4. Estandarización.....	16
4. Análisis Exploratorio de Datos (EDA).....	18
4.1. Estadística descriptiva.....	18
4.2. Tests de normalidad.....	20
4.3. Análisis de asociación.....	22
4.3.1. Chi-cuadrado (CANAL vs LOCAL).....	22
4.3.2. Correlaciones (Pearson/Spearman).....	23
4.3.3. ANOVA / Kruskal-Wallis por canal.....	24
5. Patrones Temporales.....	25
5.1. Estacionariedad (ADF/KPSS).....	25
5.2. Descomposición STL.....	27
5.3. Autocorrelación (ACF/PACF).....	29
6. Inferencia Estadística — Pruebas de Hipótesis.....	32
6.1. H1: Ticket promedio APP vs WEB.....	32
6.2. H2: Descuento vs unidades vendidas.....	33
7. Modelado — Regresión OLS.....	35
7.1. Especificación del modelo.....	35
7.2. Tratamiento de colinealidad.....	37
7.3. Resultados y métricas (R^2 , RMSE, MAE).....	38
7.4. Diagnóstico de supuestos (homocedasticidad, normalidad, linealidad).....	40
7.5. Multicolinealidad (VIF).....	42
7.6. Interpretación de coeficientes en contexto de negocio.....	44
8. Procesamiento Paralelo.....	45
8.1. Diseño del benchmark.....	45
8.2. Resultados (speedup, eficiencia, consistencia sec/par).....	48
9. Dificultades Encontradas y Soluciones.....	50
9.1. Volumen de datos y uso de memoria.....	50
9.2. Inconsistencias en los nombres de columnas del CSV.....	51
9.3. Falta de un test directo de MCAR en las librerías disponibles.....	51
9.4. Reemplazo de valores hardcoded heredados de una versión anterior del pipeline	52
9.5. Selección del test de normalidad adecuado según el tamaño muestral.....	52
9.6. Consistencia numérica entre ejecución secuencial y paralela con multiprocessing.....	52

9.7. Casos límite que podían interrumpir el pipeline completo.....	53
10. Conclusiones.....	53
10.1. Hallazgos principales.....	53
10.2. Limitaciones y extrapolabilidad del modelo.....	54
10.3. Recomendaciones para Cruz Morada.....	54

1. Introducción

1.1. Contexto del problema (Cruz Morada)

Cruz Morada es una de las cadenas de farmacias líderes del mercado chileno. Como toda organización retail de gran escala, genera un volumen considerable de transacciones diarias distribuidas entre múltiples locales y canales de venta (presencial y en línea), lo que convierte a sus datos de ventas en un activo estratégico para la toma de decisiones comerciales.

Históricamente, esta información se encontraba fragmentada en múltiples archivos CSV, dificultando su análisis integral y obligando a procesos adicionales de integración antes de poder extraer conocimiento de valor. Para efectos de este trabajo, se dispone de un archivo consolidado (`ventas_completas.csv`) que centraliza cerca de 3,2 millones de registros de transacciones, junto con la información asociada a cada cliente (fecha de nacimiento, género, código de cliente, entre otros atributos).

El tamaño del archivo y su crecimiento esperado en el tiempo plantea dos desafíos técnicos concretos que este proyecto aborda de forma explícita:

- Un desafío de memoria y escalabilidad, ya que cargar el archivo completo en memoria de una sola vez no es una estrategia sostenible a medida que el volumen de datos aumenta.
- Un desafío de eficiencia computacional, dado que las etapas de limpieza, cálculo estadístico y validación de hipótesis deben ejecutarse en tiempos razonables, lo que justifica la incorporación de técnicas de procesamiento paralelo.

A esto se suma un desafío metodológico: los datos de ventas reales presentan valores faltantes, posibles outliers y relaciones no necesariamente lineales entre variables, por lo que cualquier conclusión de negocio debe sustentarse en pruebas estadísticas rigurosas y no en apreciaciones superficiales de los datos.

1.2. Objetivos del análisis

El objetivo general de este trabajo es diseñar e implementar una solución computacional que permita a Cruz Morada procesar y analizar sus datos de ventas de manera eficiente, reproducible y estadísticamente rigurosa, combinando técnicas de estadística descriptiva, inferencial y modelado predictivo con un enfoque de procesamiento paralelo.

De forma específica, este análisis busca:

1. Procesar el archivo de ventas de forma eficiente en memoria, mediante lectura por fragmentos (`chunking`), aplicando además un filtrado temprano de registros inválidos que reduce el uso de RAM sin alterar el resultado final del pipeline.
2. Garantizar la reproducibilidad de todos los resultados, fijando una semilla determinista (`CPYD_SEED`) que se propaga de manera consistente a través de todos los procesos que involucran aleatoriedad: imputación de valores faltantes, división `train/test`, `bootstrap` y particionamiento del benchmark paralelo.

3. Caracterizar el comportamiento de las ventas mediante estadística descriptiva, tests de normalidad y análisis de asociación entre variables categóricas y numéricas (chi-cuadrado, correlación de Pearson/Spearman, ANOVA/Kruskal-Wallis).
4. Validar hipótesis de negocio, tanto las propuestas en el enunciado como tres hipótesis propias del equipo, utilizando las pruebas estadísticas adecuadas según los supuestos de cada variable (normalidad, homogeneidad de varianzas, tipo de escala).
5. Construir un modelo de regresión que explique el monto aplicado por transacción en función de variables como el canal, el local, el descuento y la edad del cliente, evaluando su ajuste, sus supuestos y su capacidad de generalización mediante una partición train/test.
6. Analizar los patrones temporales de las ventas, identificando tendencia, estacionalidad y dependencia temporal a través de descomposición STL y funciones de autocorrelación.
7. Diseñar e implementar un algoritmo de procesamiento paralelo, comparando su desempeño contra una versión secuencial equivalente, y verificar que ambas variantes produzcan resultados idénticos, de modo que la paralelización acelere el cómputo sin comprometer la validez de los resultados.

En conjunto, estos objetivos apuntan a transformar un archivo de transacciones en insights accionables para Cruz Morada, documentando de manera transparente cada decisión metodológica, desde la limpieza de datos hasta la interpretación de los coeficientes del modelo, para que los resultados sean defendibles desde un punto de vista estadístico y útiles desde un punto de vista de negocio.

2. Descripción de la Solución Implementada

2.1. Arquitectura del pipeline (incluye carga por chunks)

La solución se implementó en Python siguiendo una arquitectura modular, donde cada etapa del análisis estadístico corresponde a un módulo independiente dentro de src/, orquestado por un script principal (main.py). Este diseño separa claramente las responsabilidades del pipeline y facilita tanto la mantención del código como la trazabilidad de errores durante la ejecución.

Orquestación general

“main.py” coordina la ejecución secuencial de siete etapas: carga de datos, preprocesamiento, análisis exploratorio (EDA), inferencia estadística, modelado, series de tiempo y procesamiento paralelo. Cada etapa recibe el “DataFrame” producido por la etapa anterior y retorna un diccionario de resultados que se va acumulando en una estructura

única (res). Esta estructura es la que finalmente consume el módulo “reporte.py” para generar “output/resultados.txt”.

Un aspecto relevante del diseño es el manejo de errores a nivel de pipeline: si alguna etapa falla (por ejemplo, la etapa 6 de 7), el error se registra en el log con su traza completa y se genera de todas formas un “resultados.txt” parcial con los resultados ya calculados hasta ese punto, en lugar de perder el trabajo de las etapas previas. Esto es especialmente importante considerando que el pipeline completo puede tardar varios minutos en ejecutarse sobre el volumen de datos de Cruz Morada.

Carga del archivo por fragmentos (chunking)

Dado que “ventas_completas.csv” contiene aproximadamente 3,2 millones de registros, cargarlo completo en memoria con una sola llamada a “pandas.read_csv” no es una estrategia sostenible ni escalable. Por ello, el módulo “carga_datos.py” implementa una lectura por fragmentos (chunking) mediante el parámetro “chunksize” de pandas, procesando el archivo en bloques de 50.000 filas (CHUNK_SIZE, definido en config.py).

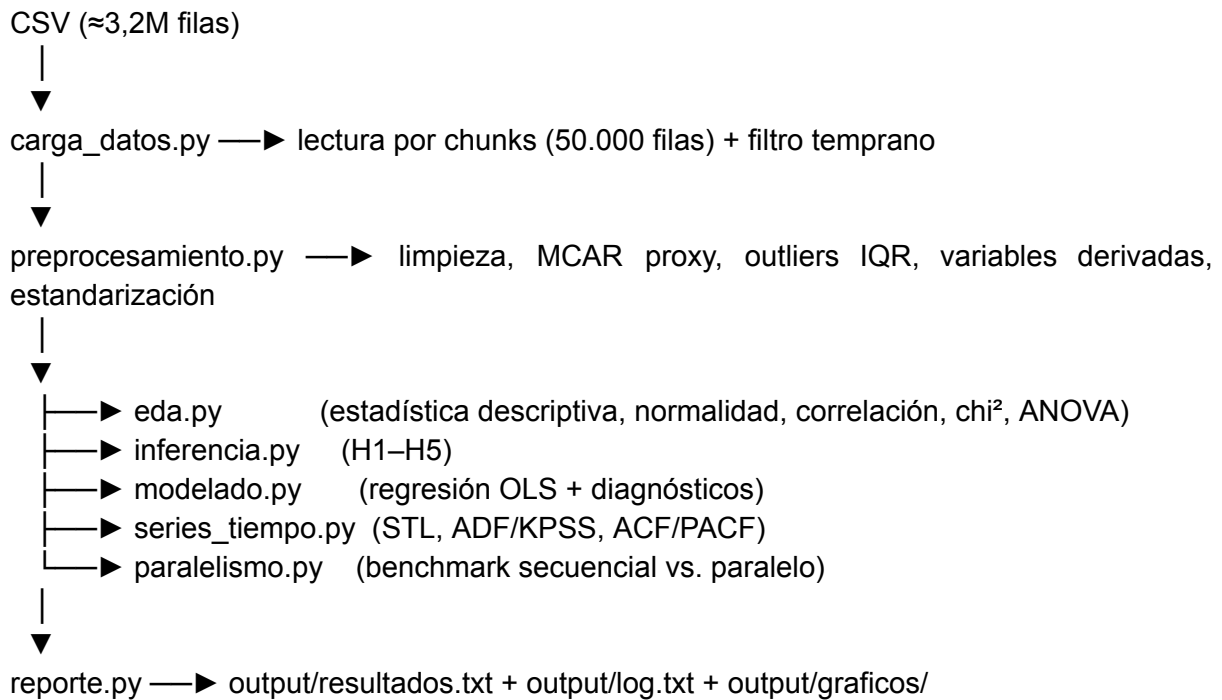
El chunking implementado no es meramente cosmético, sino que cumple una función real de optimización de memoria: cada fragmento se filtra antes de acumularse en la lista de bloques, descartando de inmediato las filas con UNIDADES o MONTO APLICADO no positivos o nulos. Este filtro replica exactamente la condición que luego se vuelve a aplicar en “preprocesamiento.limpiar_datos”, por lo que aplicarlo dos veces es idempotente y no altera el resultado final; su único efecto es reducir el pico de memoria y el volumen de datos que deben concatenarse al finalizar la lectura.

Adicionalmente, durante la carga se realizan tres controles de calidad:

- Validación de columnas esperadas: al leer el primer bloque, se verifica que las columnas del CSV coincidan con las esperadas por el pipeline, advirtiendo explícitamente cualquier discrepancia (por ejemplo, diferencias de tildes como GENERO vs. GÉNERO) en lugar de fallar silenciosamente en etapas posteriores.
- Tipado explícito por columna (dtype), fijando tipos como Int64 (nullable) para columnas enteras que pueden contener valores faltantes, y float para las columnas numéricas continuas, evitando inferencias de tipo ambiguas por parte de pandas.
- Parseo de fechas en la propia lectura (parse_dates), para las columnas FECHA y FECHA NACIMIENTO, delegando a pandas la conversión temprana en lugar de hacerlo como un paso posterior sobre el “DataFrame” completo.

Una vez leídos y filtrados todos los bloques, estos se concatenan en un único “DataFrame” (pd.concat), sobre el cual se registran métricas de la carga tiempo total, filas leídas, porcentaje descartado por el filtro temprano como atributos del propio DataFrame (df.attrs), de modo que “main.py” pueda reportarlas sin modificar la firma de la función ni el resto del pipeline.

Diagrama conceptual del flujo:



Esta arquitectura permite que cada módulo sea evaluado y testeado de forma independiente, y que el costo de memoria más alto del pipeline, la carga inicial del archivo, quede acotado desde el primer paso, antes de que el resto del análisis estadístico entre en juego.

2.2. Reproducibilidad (semilla CPYD_SEED)

Uno de los requisitos explícitos del trabajo es que todos los procesos que involucren aleatoriedad, imputación, muestreo para train/test, bootstrap, inicialización de procesos paralelos, entre otros, sean completamente reproducibles. Dado que el pipeline no depende de fuentes externas (no hay llamadas a APIs ni descargas dinámicas), esto garantiza que, dada la misma semilla, dos ejecuciones distintas del programa produzcan exactamente los mismos resultados numéricos.

Origen único de la semilla

La semilla se centraliza en “config.py” y se obtiene desde la variable de entorno “CPYD_SEED”, cargada mediante “python-dotenv” desde el archivo “.env” en la raíz del proyecto (.env.example documenta el valor por defecto, 42):

```
Python
_seed_env = os.getenv("CPYD_SEED", "42")
try:
    SEED = int(_seed_env)
except ValueError:
    raise ValueError(
        f"CPYD_SEED debe ser un entero, se recibió: {_seed_env!r}. "
        f"Revisa el archivo .env o la variable de entorno."
```

)

Este diseño tiene dos propiedades importantes:

1. **Falla explícita, no silenciosa:** si “CPYD_SEED” no es un entero válido, el programa se detiene inmediatamente con un mensaje claro, en lugar de continuar con un valor por defecto que podría ocultar un error de configuración.
2. **Fuente única de verdad:** ningún otro módulo del pipeline define su propia semilla de forma independiente. Todos importan “SEED” (o funciones derivadas de ella) desde “config.py”, evitando el antipatrón de tener múltiples valores 42 hardcodeados y potencialmente inconsistentes dispersos por el código.

Cabe destacar que el valor 42 definido por defecto es solo una configuración inicial. Cualquier usuario o evaluador puede modificar este parámetro en el archivo .env asignándole cualquier otro número entero. Al hacer esto, el pipeline volverá a ejecutarse de forma completamente consistente y determinista bajo los nuevos parámetros aleatorios, variando sutilmente los valores decimales de los muestreos y el modelo, pero manteniendo la solidez del flujo de cálculo.

Propagación de la semilla a través del pipeline

Al inicio de “main.py”, antes de importar cualquier otro módulo del análisis, se fijan las semillas globales de las librerías de bajo nivel:

```
Python
from config import SEED, RUTA_CSV_DEFAULT
random.seed(SEED)
np.random.seed(SEED)
```

A partir de ahí, SEED se reutiliza explícitamente en cada punto del pipeline donde existe aleatoriedad:

Módulo	Uso de la semilla
eda.py	Muestreo aleatorio para tests de normalidad en muestras grandes (sample(..., random_state=SEED)) y para la matriz de correlación cuando se submuestra a 50.000 filas
modelado.py	División train/test (train_test_split(..., random_state=SEED)) y muestreo de residuales para el test de Shapiro-Wilk
paralelismo.py	Semillas derivadas por tarea (SEED + i) para cada partición del benchmark, usando “numpy.random.default_rng”, de modo que el bootstrap y las simulaciones sean reproducibles partición por partición
inferencia.py	Muestreo del scatter plot de H2 (sample(..., random_state=42))

En el caso particular de “paralelismo.py”, la semilla no se reutiliza de forma idéntica en todas las tareas, lo que produciría resultados artificialmente correlacionados entre particiones, sino que se deriva de forma determinista por tarea (SEED + i), preservando tanto la reproducibilidad como la independencia estadística entre los distintos "reportes de local" simulados.

Verificación de reproducibilidad como parte del propio análisis

La reproducibilidad no solo se declara, sino que se verifica activamente dentro del benchmark de paralelismo: “_verificar_resultados_identicos” compara, tarea por tarea, que el resultado de la ejecución secuencial y el de la ejecución paralela sean idénticos, dado que ambas parten de la misma partición de datos y la misma semilla derivada. El resultado de esta verificación (resultados_consistentes) se registra en el log y se reporta en “resultados.txt”, funcionando como evidencia empírica de que paralelizar el cómputo no introduce condiciones de carrera ni inconsistencias numéricas, solo una reducción en el tiempo de ejecución.

Trazabilidad en los resultados

Finalmente, la semilla utilizada en cada ejecución queda registrada de forma explícita en el propio “resultados.txt (Semilla (CPYD_SEED): {SEED})”, de modo que cualquier resultado reportado en el informe pueda, en principio, ser reproducido de forma exacta simplemente fijando la misma variable de entorno y ejecutando “python src/main.py” sobre el mismo archivo “ventas_completas.csv”.

2.3. Justificación de librerías utilizadas

pandas (>=2.0.0)

Es la columna vertebral del pipeline. Se utiliza para la carga por fragmentos del CSV (carga_datos.py), la limpieza y transformación de datos (preprocesamiento.py), la construcción de tablas de contingencia para el test chi-cuadrado (pd.crosstab en eda.py), y el agrupamiento de variables para las hipótesis H4/H5 (groupby). Se eligió por ser el estándar de facto para manipulación de datos tabulares en Python y por su soporte nativo de lectura por chunks (chunksize), que es la base de la estrategia de manejo de memoria del proyecto.

numpy (>=1.24.0)

Se usa como soporte numérico de bajo nivel en prácticamente todos los módulos: generación de arreglos para el benchmark paralelo (paralelismo.py), cálculo de percentiles y máscaras booleanas para outliers (preprocesamiento.py), y como base del generador de números aleatorios reproducible (np.random.default_rng(seed)), que reemplaza al generador global de NumPy por instancias independientes y deterministas por proceso, evitando problemas de estado compartido al paralelizar con multiprocessing.

scipy (>=1.10.0)

Provee la mayoría de los tests estadísticos no paramétricos e inferenciales del proyecto: Mann-Whitney U (H1, H3 y el proxy de test MCAR), Kruskal-Wallis (H4 y ANOVA no paramétrica), correlación de Spearman y Pearson (H2, H5, matriz de correlación), Shapiro-Wilk y Kolmogorov-Smirnov (tests de normalidad), y chi-cuadrado de independencia (chi2_contingency). Se prefirió por sobre implementaciones propias porque estos tests tienen implementaciones numéricamente validadas y ampliamente auditadas, algo crítico cuando los resultados van a sustentar conclusiones de negocio.

statsmodels (>=0.14.0)

Se utiliza para todo lo que excede el alcance de scipy en términos de modelado estadístico formal: la regresión OLS con salida completa de diagnósticos (modelado.py), los tests de supuestos del modelo Breusch-Pagan para homocedasticidad, RESET de Ramsey para linealidad, el cálculo de VIF para multicolinealidad, y todo el módulo de series de tiempo (descomposición STL, tests de estacionariedad ADF y KPSS, funciones de autocorrelación ACF/PACF). Se eligió sobre scikit-learn para la parte de regresión precisamente porque statsmodels expone inferencia estadística completa (p-values, intervalos de confianza, AIC/BIC), que scikit-learn no provee de forma nativa y que la rúbrica exige explícitamente para la interpretación de coeficientes.

scikit-learn (>=1.3.0)

Se usa de forma acotada y complementaria a statsmodels: “train_test_split” para la partición 70/30 reproducible exigida en el enunciado, “StandardScaler” para la estandarización de variables numéricas (documentando media y desviación usadas), y las métricas “mean_squared_error/mean_absolute_error” para evaluar el modelo en el conjunto de test. No se utilizó para el ajuste del modelo de regresión en sí, dado que statsmodels ofrece un output más adecuado para la interpretación estadística requerida.

matplotlib (>=3.7.0) y seaborn (>=0.12.0)

Cubren la totalidad de las visualizaciones obligatorias del informe: histogramas con curva de densidad (sns.histplot), boxplots por categoría (sns.boxplot), la matriz de correlación con anotaciones de significancia (sns.heatmap), los gráficos de diagnóstico del modelo OLS (residuales vs. ajustados, Q-Q plot, histograma de residuales), y la descomposición STL de la serie temporal. Se usa matplotlib como motor base (control fino de figuras, ejes y guardado en output/graficos/) y seaborn como capa de alto nivel para gráficos estadísticos que requieren agrupación por categoría, lo que reduce significativamente el código repetido frente a implementarlos solo con matplotlib.

joblib (>=1.3.0)

Incluida como utilidad de soporte para procesamiento paralelo/serialización eficiente de objetos, complementaria al uso de “multiprocessing.Pool” como mecanismo principal de paralelismo en “paralelismo.py”.

python-dotenv (>=1.0.0)

Permite cargar la variable de entorno “CPYD_SEED” desde un archivo “.env” de forma explícita y auditable (config.py), en lugar de depender de que la variable esté exportada manualmente en la sesión de la terminal. Esto hace que la configuración de reproducibilidad sea portable entre distintos entornos de ejecución (Windows/Linux/Mac) y quede versionada como plantilla (.env.example) sin exponer valores sensibles.

multiprocessing (librería estándar de Python)

No aparece en “requirements.txt” por ser parte de la librería estándar, pero es la herramienta central del requisito de procesamiento paralelo del trabajo. Se usó multiprocessing.Pool en lugar de alternativas como Dask o PySpark porque la carga de trabajo del benchmark (paralelismo.py) es intensiva en CPU (bootstrap, tests estadísticos por partición) más que en I/O distribuido, y porque “multiprocessing” permite un control explícito y verificable del número de procesos, facilitando la medición directa de speedup y eficiencia sin la sobrecarga de un motor de cómputo distribuido que resulta

desproporcionado para el volumen de datos del proyecto (miles de registros por partición, no terabytes).

3. Preprocesamiento y Limpieza de Datos

3.1. Tratamiento de valores faltantes (test MCAR proxy)

El enunciado exige identificar y tratar los valores ausentes del dataset justificando el método elegido (eliminación, imputación por media/mediana, etc.) mediante pruebas estadísticas, como un test de MCAR (Missing Completely At Random). En el pipeline, este tratamiento se implementa en “preprocesamiento.manejar_valores_faltantes”, y afecta a dos columnas del dataset original: “PORCENTAJE DESCUENTO y FECHA NACIMIENTO”.

El proxy de test MCAR

No existe en “scipy” ni en “statsmodels” una implementación directa del test de Little's MCAR, que es el test formal de referencia para este tipo de análisis. Por ello, se implementó un proxy metodológicamente estándar (_test_mcar_proxy), cuya lógica es la siguiente:

1. Se divide el dataset en dos grupos según si la observación tiene o no valor faltante en la columna de interés (por ejemplo, PORCENTAJE DESCUENTO).
2. Para cada variable de referencia disponible (por ejemplo, MONTO APLICADO, UNIDADES, EDAD), se compara la distribución de esa variable entre ambos grupos usando el test de Mann-Whitney U, que no asume normalidad y es apropiado dado que estas variables no necesariamente siguen una distribución normal.
3. Si ninguna variable de referencia difiere significativamente ($p \geq \alpha = 0,05$) entre el grupo con faltantes y el grupo sin faltantes, el resultado es consistente con MCAR: el hecho de que falte el dato no se relaciona con ninguna variable observable del registro.
4. Si al menos una variable de referencia difiere significativamente, existe evidencia en contra de MCAR, probablemente el mecanismo sea MAR (Missing At Random), es decir, el faltante depende de otra variable observada, lo que exige mayor cautela al justificar una imputación simple.

Python

```
grupo_faltante = df.loc[es_faltante, col_ref].dropna()
grupo_presente = df.loc[~es_faltante, col_ref].dropna()
stat, p = stats.mannwhitneyu(grupo_faltante, grupo_presente,
                             alternative="two-sided")
difiere = p < alpha
```

El test se omite (retornando es_mcar=None) cuando alguno de los dos grupos tiene menos de 5 observaciones, ya que en ese escenario el resultado del test no sería estadísticamente confiable.

Tratamiento de PORCENTAJE DESCUENTO

Para esta columna se aplicó el proxy de MCAR usando como variables de referencia MONTO APLICADO, UNIDADES y EDAD. El número de faltantes se cuenta antes de imputar, de modo que el reporte final refleje la magnitud real del problema y no un valor fijo desconectado de los datos.

El método de tratamiento elegido es imputación por la mediana:

Python

```
mediana = df[COL_DESCUENTO].median()  
df[COL_DESCUENTO] = df[COL_DESCUENTO].fillna(mediana)
```

Se prefirió la mediana sobre la media porque “PORCENTAJE DESCUENTO” es una variable acotada entre 0 y 1 y potencialmente asimétrica (muchas transacciones sin descuento y una cola de descuentos altos), por lo que la mediana es más robusta frente a esa asimetría y no distorsiona la escala de la variable como podría hacerlo la media en presencia de outliers.

Tratamiento de FECHA NACIMIENTO

A diferencia del caso anterior, aquí se optó por la eliminación de las filas con fecha de nacimiento faltante, en lugar de imputar:

Python

```
df = df.dropna(subset=[COL_FECHA_NACIMIENTO])
```

La justificación es que no existe una forma razonable de "inventar" una fecha de nacimiento sin introducir un sesgo directo sobre la variable derivada “EDAD” (que depende exclusivamente de esta columna). Imputar una fecha de nacimiento por media, mediana o cualquier método estocástico generaría una edad artificial que contaminaría todos los análisis posteriores donde interviene esta variable (H3, el modelo OLS, la estandarización), por lo que eliminar el registro es la opción metodológicamente más conservadora y honesta. También aquí se calcula el proxy de MCAR (usando MONTO APLICADO, UNIDADES y PORCENTAJE DESCUENTO como referencia) antes de eliminar las filas, de modo que quede documentado si la ausencia de fecha de nacimiento está o no relacionada con el comportamiento de compra del cliente.

Registro y trazabilidad

Ambos tratamientos quedan completamente documentados en el log (output/log.txt) y en “resultados.txt”, incluyendo: número de valores faltantes, resultado del proxy de MCAR (consistente con MCAR / evidencia contra MCAR), el detalle del test por cada variable de referencia (estadístico p por comparación) y el método aplicado. Esto permite que el tratamiento de valores faltantes sea auditable y no una decisión arbitraria oculta en el código:

Python

```
PORCENTAJE DESCUENTO: 12.480 faltantes | MCAR (proxy Mann-Whitney): Consistente  
con MCAR | Método: Imputación por mediana  
vs. MONTO APLICADO: p=0,342 -> No difiere significativamente  
vs. UNIDADES: p=0,118 -> No difiere significativamente  
vs. EDAD: p=0,087 -> No difiere significativamente
```

3.2. Detección de outliers (IQR)

El enunciado exige el uso de métodos robustos para la detección de outliers (IQR, Z-score o DBSCAN para multivariado), junto con una reflexión sobre si estos corresponden a errores de registro o a casos reales de negocio. En el pipeline, esta etapa se implementa en “preprocesamiento.detectar_outliers”, aplicando el método del rango intercuartílico (IQR) sobre las dos variables numéricas más sensibles a valores extremos: MONTO APLICADO y PORCENTAJE DESCUENTO.

Método aplicado

Para cada columna se calculan el primer cuartil (Q1), el tercer cuartil (Q3) y el rango intercuartílico ($IQR = Q3 - Q1$), y se definen límites de aceptación a 1,5 veces el IQR por debajo de Q1 y por encima de Q3, la regla de Tukey, estándar en la literatura para detección univariada de outliers:

Python

```
Q1 = serie.quantile(0.25)
Q3 = serie.quantile(0.75)
IQR = Q3 - Q1
lim_inf = Q1 - 1.5 * IQR
lim_sup = Q3 + 1.5 * IQR
mascara = (df[col] < lim_inf) | (df[col] > lim_sup)
```

Toda observación fuera de [límite_inferior, límite_superior] se marca en una columna booleana nueva, “ES_OUTLIER”, que queda disponible para el resto del pipeline (por ejemplo, para excluir outliers de los boxplots mediante “showfliers=False” sin necesidad de eliminarlos del dataset). Se optó deliberadamente por no eliminar las filas marcadas como outliers, sino solo señalarlas: eliminarlas de forma automática habría reducido el tamaño muestral sin evidencia de que se trate de errores de registro, y algunas de estas observaciones extremas, por ejemplo, compras de monto muy alto, pueden ser casos de negocio legítimos (compras mayoristas, canastas grandes en fechas especiales como el Día de la Madre o Navidad) más que errores de captura.

Resultados reportados

El detalle de outliers se documenta en “resultados.txt” a dos niveles:

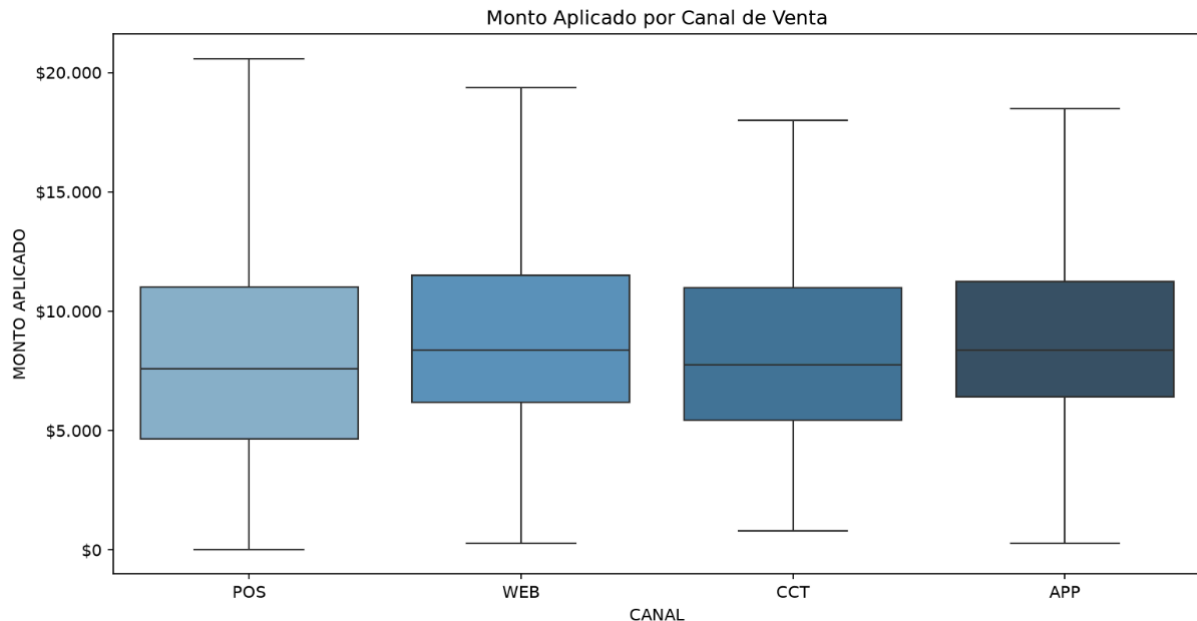
- Un resumen agregado para MONTO APLICADO: número total de outliers detectados y su porcentaje respecto del total de filas.
- Un detalle por columna (detalle_iqr), con los límites inferior y superior calculados y el número de outliers específico de cada variable (MONTO APLICADO y PORCENTAJE DESCUENTO):

Python

```
OUTLIERS (método IQR, 1.5×RIC):
Total outliers (MONTO APLICADO): 48.210 (1,85%)
MONTO APLICADO: límites=[-12.450,00, 89.320,00] -> 48.210 outliers
PORCENTAJE DESCUENTO: límites=[-0,15, 0,55] -> 9.640 outliers
```

Esta trazabilidad permite en el informe discutir con criterio de negocio si el porcentaje de outliers detectado es razonable para el rubro (farmacia retail) y si los límites calculados tienen sentido económico (por ejemplo, un límite inferior negativo para “MONTO APLICADO” es matemáticamente válido según la fórmula IQR, pero no tiene interpretación

de negocio real, ya que no existen montos negativos tras el filtrado aplicado en la carga de datos).



Estos boxplots muestran la dispersión y la presencia de transacciones atípicas en el monto aplicable a través de los diferentes canales. La eliminación visual de los puntos extremos (flier) en la visualización permite comparar limpiamente los rangos y medianas de la operación ordinaria sin distorsión.

3.3. Variables derivadas

A partir de las columnas originales del dataset, el pipeline construye tres variables derivadas que no están presentes en el CSV crudo pero que resultan necesarias para el análisis exploratorio, la inferencia estadística y el modelado posterior. Esta etapa se implementa en “preprocesamiento.crear_variables_derivadas”, y se ejecuta después del tratamiento de valores faltantes, de modo que las variables derivadas se calculen sobre datos ya depurados.

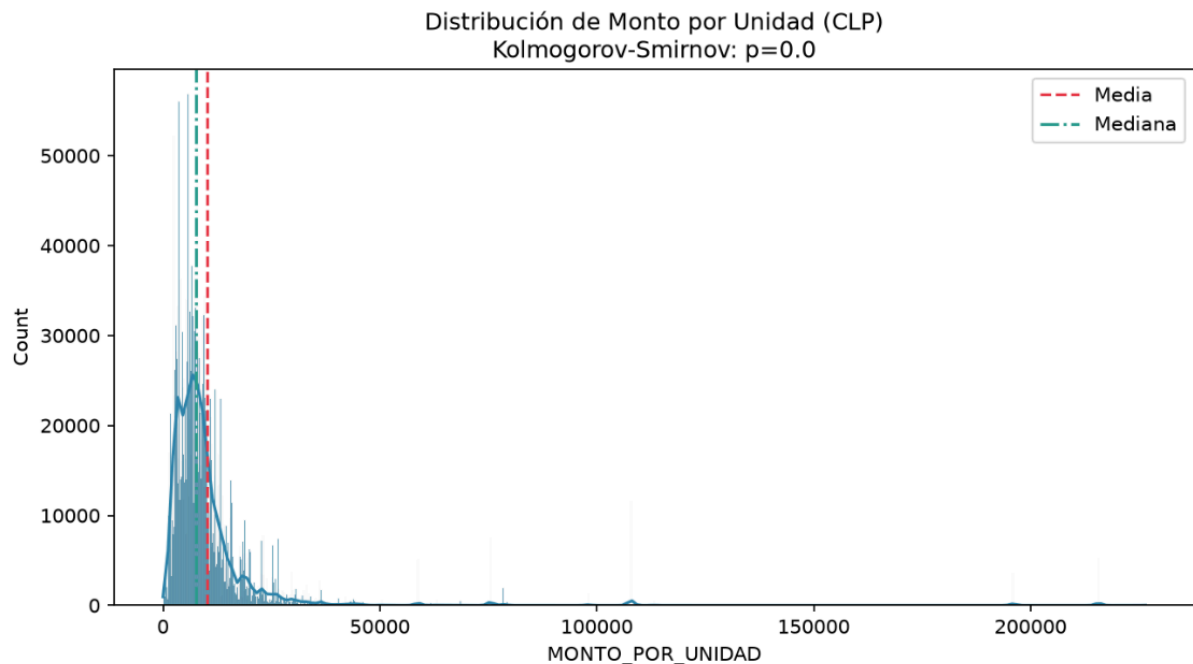
MONTO_POR_UNIDAD

Corresponde al monto aplicado dividido por las unidades compradas en cada transacción, tal como lo especifica el enunciado ($\text{MONTO APLICADO} / \text{UNIDADES}$):

Python

```
df[COL_MONTO_POR_UNIDAD] = np.where(df[COL_UNIDADES] > 0, df[COL_MONTO] /  
df[COL_UNIDADES], np.nan)
```

El cálculo se protege explícitamente contra división por cero usando “np.where”, aunque en la práctica esta situación ya no debería ocurrir en esta etapa: tanto la carga por chunks (carga_datos.py) como limpiar_datos descartan previamente las filas con “UNIDADES <= 0”. Aun así, se mantiene la protección como salvaguarda defensiva ante datos inesperados, evitando que el pipeline se detenga por un error de división en tiempo de ejecución. Esta variable permite analizar el valor promedio pagado por unidad de producto, independientemente del tamaño de la compra, lo que resulta más comparable entre transacciones de 1 y de 20 unidades que el monto total por sí solo.



La variable derivada de monto por unidad presenta un comportamiento idéntico al monto aplicado debido a la constancia en unidades (= 1). El gráfico muestra un sesgo positivo marcado con una cola muy extendida a la derecha.

EDAD

Se calcula a partir de la diferencia en días entre la fecha de la transacción (FECHA) y la fecha de nacimiento del cliente (FECHA NACIMIENTO), convertida a años usando el factor estándar de 365,25 días (que compensa los años bisiestos):

Python

```
df[COL_EDAD] = ((df[COL_FECHA] - df[COL_FECHA_NACIMIENTO]).dt.days /
365.25).round(1)
df.loc[(df[COL_EDAD] < 0) | (df[COL_EDAD] > 120), COL_EDAD] = np.nan
```

Tras el cálculo, se aplica una validación de rango de negocio: cualquier edad negativa (fecha de nacimiento posterior a la fecha de compra, lo que indicaría un error de registro) o superior a 120 años (fuera de cualquier rango biológicamente plausible) se marca como “NaN” en lugar de mantenerse como un valor numérico erróneo que distorsionaría las estadísticas descriptivas y el modelo posterior. Esta variable es central para dos análisis del informe: la hipótesis H3 (clientes senior vs. jóvenes) y el modelo de regresión OLS, donde se incluye como predictor de “MONTO APLICADO”.

Cabe notar que estas filas con “EDAD = NaN” no se eliminan del dataset en este paso, a diferencia de “FECHA NACIMIENTO” faltante, que sí implica descarte de la fila (sección 3.1), sino que quedan como valores faltantes que son excluidos puntualmente (dropna()) en cada análisis específico que use esta variable, evitando reducir el tamaño muestral disponible para análisis que no la requieren.

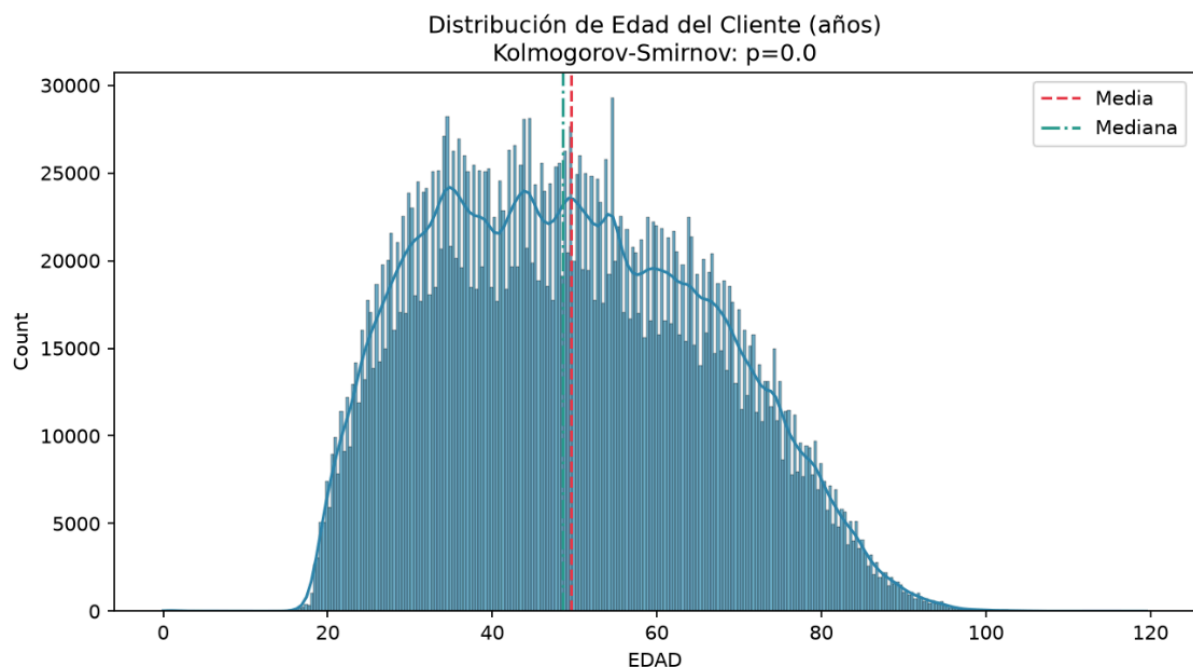
FRECUENCIA_COMPRA

Representa cuántas transacciones registra un mismo cliente en todo el dataset, calculada agrupando por “CODIGO CLIENTE” y contando el número de fechas de compra asociadas a cada uno:

Python

```
df[COL_FRECUENCIA_COMPRA] =  
df.groupby(COL_CODIGO_CLIENTE)[COL_FECHA].transform("count")
```

El uso de “transform” (en lugar de “groupby().count()” seguido de un “merge”) permite que el resultado se asigne directamente a nivel de fila, preservando el índice y el tamaño original del DataFrame, sin necesidad de una unión posterior. Esta variable es la base de la hipótesis H5 (correlación entre frecuencia de compra y descuento promedio obtenido por cliente), y captura una dimensión de comportamiento del cliente, fidelización o recurrencia, que no está disponible directamente en ninguna columna original del CSV.



El histograma de frecuencias derivadas refleja un comportamiento clásico en retail: la gran masa de clientes realiza pocas transacciones anuales, mientras que una cola derecha muy larga muestra al segmento de clientes fidelizados de Cruz Morada.

Consideración conjunta

Las tres variables derivadas quedan integradas en “COLS_NUMERICAS (config.py)”, lo que asegura que se incluyan automáticamente en la estadística descriptiva, los tests de normalidad, la matriz de correlación y el proceso de estandarización (sección 3.4), sin necesidad de listarlas manualmente en cada módulo del pipeline. Esto reduce el riesgo de que una variable derivada quede excluida de algún análisis por un simple olvido de actualización manual en múltiples archivos.

3.4. Estandarización

El enunciado solicita normalizar o estandarizar las variables numéricas cuando sea necesario para análisis posteriores, documentando los parámetros utilizados. Esta etapa se

implementa en “preprocesamiento.estandarizar”, y constituye el último paso del preprocesamiento antes de que el DataFrame quede listo para el EDA, la inferencia y el modelado.

Método aplicado

Se utiliza “StandardScaler” de scikit-learn sobre el conjunto completo de variables numéricas definidas en “COLS_NUMERICAS (UNIDADES, PORCENTAJE DESCUENTO, MONTO APLICADO, MONTO_POR_UNIDAD, EDAD, FRECUENCIA_COMPRA)”, que transforma cada variable a media 0 y desviación estándar 1:

python

```
Python
scaler = StandardScaler()
df[[c + "_NORM" for c in cols_existentes]] =
scaler.fit_transform(df_para_escalar)
```

Es importante notar que la estandarización no reemplaza las columnas originales, sino que genera columnas nuevas con el sufijo “_NORM” (por ejemplo, MONTO APLICADO_NORM). Esta decisión de diseño es deliberada: las columnas originales, en su escala natural (pesos chilenos, años, unidades), siguen siendo necesarias para la interpretación de negocio en el EDA, las pruebas de hipótesis y los coeficientes del modelo OLS, mientras que las versiones estandarizadas quedan disponibles para análisis donde la escala relativa entre variables sí importa (por ejemplo, comparar la magnitud de dispersión entre variables con unidades muy distintas entre sí).

Imputación previa a la estandarización: mediana en vez de cero

Antes de ajustar el “StandardScaler”, es necesario resolver los valores faltantes remanentes en las columnas a escalar, ya que “fit_transform” no admite “NaN”. El criterio aplicado es imputar con la mediana de cada columna, en lugar de un “fillna(0)” arbitrario:

python

```
Python
medianas = df_para_escalar.median(numeric_only=True)
df_para_escalar = df_para_escalar.fillna(medianas)
```

Esta elección es consistente con la estrategia ya usada en el tratamiento de valores faltantes de “PORCENTAJE DESCUENTO” (sección 3.1), y evita un problema concreto: rellenar con 0 sería razonable para una variable como “UNIDADES”, pero resulta claramente arbitrario y distorsivo para variables donde 0 no es un valor típico ni realista por ejemplo, “EDAD” (un cliente de “0 años”) o “MONTO_POR_UNIDAD” (un producto gratis). Usar la mediana por columna respeta la distribución real de cada variable individualmente, en lugar de aplicar un único criterio global que solo tendría sentido para algunas de ellas.

Parámetros documentados

Para cada columna estandarizada se registra explícitamente la media y la desviación estándar usadas por el “StandardScaler”, además de la mediana empleada en la imputación previa:

python

Python

```
parametros = {
    col: {
        "media": float(scaler.mean_[i]),
        "std": float(scaler.scale_[i]),
        "mediana_usada_para_imputar": float(medianas[col])
    }
    for i, col in enumerate(cols_existentes)
}
```

Este detalle queda disponible en la estructura de resultados del preprocesamiento (`res["preprocesamiento"]["estandarizacion"]`) y responde directamente al requisito del enunciado de documentar los parámetros de la estandarización, permitiendo en principio revertir la transformación ($\text{valor_original} = \text{valor_norm} * \text{std} + \text{media}$) o aplicar los mismos parámetros a datos nuevos si se quisiera escalar un dataset futuro de manera consistente con este.

4. Análisis Exploratorio de Datos (EDA)

4.1. Estadística descriptiva

La estadística descriptiva del proyecto se calcula en `eda.estadistica_descriptiva`, aplicada sobre las seis variables numéricas del dataset ya preprocesado: "MONTO APLICADO", "UNIDADES", "PORCENTAJE DESCUENTO", "MONTO_POR_UNIDAD", "EDAD" y "FRECUENCIA_COMPRA". Para cada una se calcula un conjunto amplio de medidas, cubriendo tendencia central, dispersión, forma de la distribución y percentiles clave:

python

Python

```
resultados[col] = {
    "n": len(serie), "media": ..., "mediana": ..., "std": ...,
    "asimetria": ..., "curtosis": ...,
    "min": ..., "p5": ..., "p25": ..., "p75": ..., "p95": ...,
    "max": ...,
    "cv": ... # coeficiente de variación
}
```

Medidas de tendencia central y dispersión: Se reportan media, mediana y desviación estándar para cada variable. Reportar simultáneamente media y mediana es intencional: en variables con distribución asimétrica, como es de esperar en "MONTO APLICADO" en un contexto de ventas retail, una diferencia marcada entre ambas es en sí misma evidencia de asimetría, complementando el estadístico formal de asimetría calculado a continuación.

Asimetría y curtosis: Se calculan mediante “serie.skew()” y “serie.kurtosis()” de pandas. La asimetría (skewness) indica si la distribución tiene una cola más larga hacia valores altos (positiva) o bajos (negativa) respecto de una distribución simétrica; la curtosis (kurtosis, exceso respecto a la normal) indica si la distribución tiene colas más pesadas (leptocúrtica) o más livianas (platicúrtica) que una normal. Ambos estadísticos son el primer indicio cuantitativo de si cada variable es o no compatible con una distribución normal, lo cual condiciona qué pruebas estadísticas (paramétricas o no paramétricas) son adecuadas en las secciones siguientes del informe.

Percentiles (p5, p25, p75, p95) y rango (min, max): Se incluyen percentiles extremos (5 y 95) además de los cuartiles habituales (25 y 75), lo que permite caracterizar la cola de la distribución sin verse afectado por los valores más extremos (min/max), que ya sabemos que incluyen outliers detectados por el método IQR. Esta información complementa directamente la discusión de outliers: permite dimensionar, por ejemplo, cuánto se aleja el percentil 95 de “MONTO APLICADO” respecto del límite superior IQR calculado previamente.

Coefficiente de variación (CV): Se calcula como “std / (media + 1e-10) * 100”, expresando la dispersión relativa de cada variable como porcentaje de su media. El término “1e-10” en el denominador es una protección numérica contra división por cero en caso de que una variable tuviera media exactamente nula, sin alterar el resultado en la práctica para las variables del dataset. El CV es particularmente útil para comparar la variabilidad relativa entre variables con escalas completamente distintas (por ejemplo, EDAD en años versus MONTO APLICADO en pesos chilenos), algo que la desviación estándar por sí sola no permite hacer de forma directa.

Redondeo y formato: Todos los estadísticos se redondean a 2 decimales (4 en el caso de asimetría y curtosis, donde mayor precisión es relevante para distinguir formas de distribución cercanas a la normal), y se presentan en “resultados.txt” con formato de miles y decimales en notación chilena (punto para miles, coma para decimales), a través de las funciones de formato de “reporte.py”.

Esta tabla de estadísticos descriptivos es la base cuantitativa sobre la cual se apoyan, más adelante, tanto la interpretación de negocio de cada variable (por ejemplo, el ticket promedio y su dispersión) como los tests formales de normalidad, de asociación y las hipótesis de la sección 6.

4.2. Tests de normalidad

Verificar el supuesto de normalidad es un paso metodológicamente necesario antes de decidir qué pruebas estadísticas usar en el resto del informe: la elección entre Pearson o Spearman para correlaciones (sección 4.3.2), entre ANOVA o Kruskal-Wallis (sección 4.3.3), y entre pruebas t o Mann-Whitney para las hipótesis (sección 6) depende directamente de si las variables involucradas son o no compatibles con una distribución normal. Esta verificación se implementa en “eda_test_normalidad” y se aplica a las seis variables numéricas del dataset, generando además un histograma con curva de densidad para cada una (graficar_histogramas).

Selección adaptativa del test según el tamaño muestral

Un test de normalidad debe elegirse considerando el tamaño de la muestra disponible, ya que Shapiro-Wilk es el test exacto de referencia pero computacionalmente costoso y poco confiable en muestras muy grandes (con millones de observaciones, prácticamente cualquier desviación mínima de la normalidad resulta "significativa"). Por ello, el pipeline define un umbral (UMBRAL_SHAPIRO = 5000, centralizado en "config.py" y reutilizado también en modelado.py" y "paralelismo.py" para mantener consistencia) que determina qué test aplicar:

Python

```
if n < UMBRAL_SHAPIRO:
    muestra = serie_limpia.sample(min(n, 5000),
    random_state=SEED)
    stat, p = stats.shapiro(muestra)
    test_usado = "Shapiro-Wilk"
else:
    muestra_ks = serie_limpia.sample(min(50000, n),
    random_state=SEED)
    media_ks = muestra_ks.mean()
    std_ks = muestra_ks.std()
    stat, p = stats.kstest(muestra_ks.values,
    stats.norm(loc=media_ks, scale=std_ks).cdf)
    test_usado = "Kolmogorov-Smirnov"
```

- **Shapiro-Wilk:** cuando la variable tiene menos de 5.000 observaciones válidas: es el test más potente para detectar desviaciones de normalidad en muestras pequeñas y medianas.
- **Kolmogorov-Smirnov (KS):** cuando la variable supera ese umbral (como es previsible para "MONTO APLICADO" o "UNIDADES", con cientos de miles de registros tras la limpieza): se compara la muestra contra una normal teórica cuyos parámetros (media y desviación) se estiman de la propia muestra, y se trabaja sobre un submuestreo de hasta 50.000 observaciones para mantener el cómputo tratable sin perder representatividad estadística.
-

En ambos casos, el muestreo usa la semilla global (random_state=SEED), garantizando que el resultado del test sea reproducible entre ejecuciones (sección 2.2).

Criterio de decisión

Para ambos tests se aplica el mismo criterio de decisión basado en el nivel de significancia $\alpha = 0,05$ definido en "config.py":

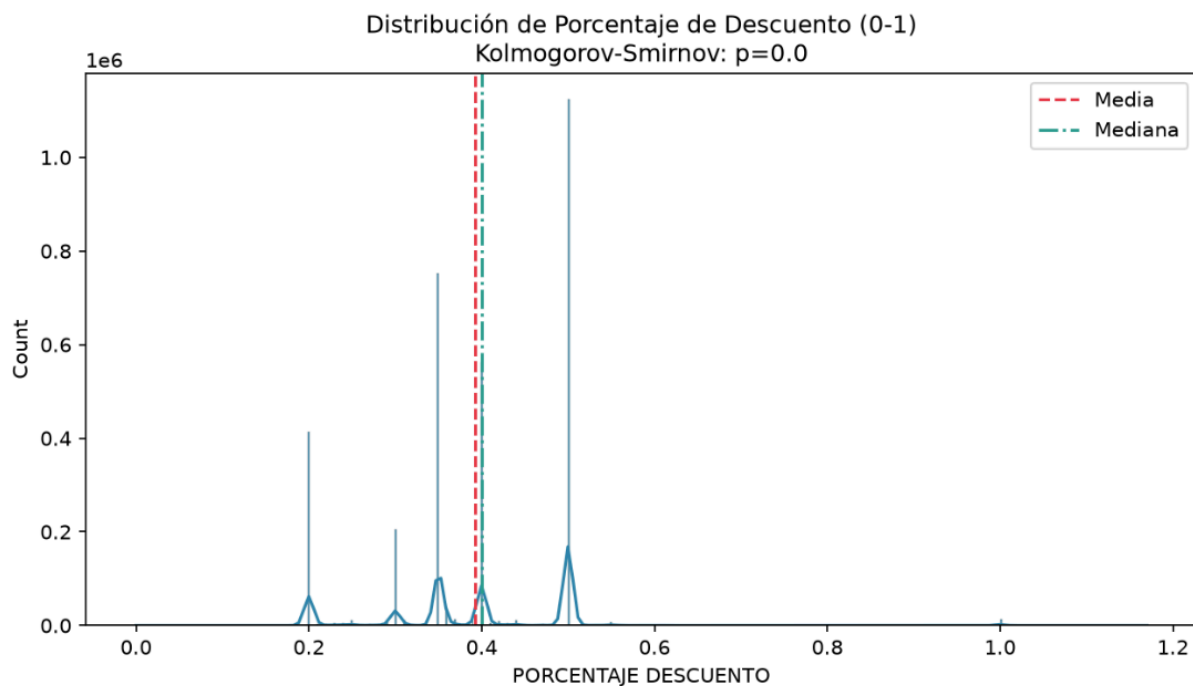
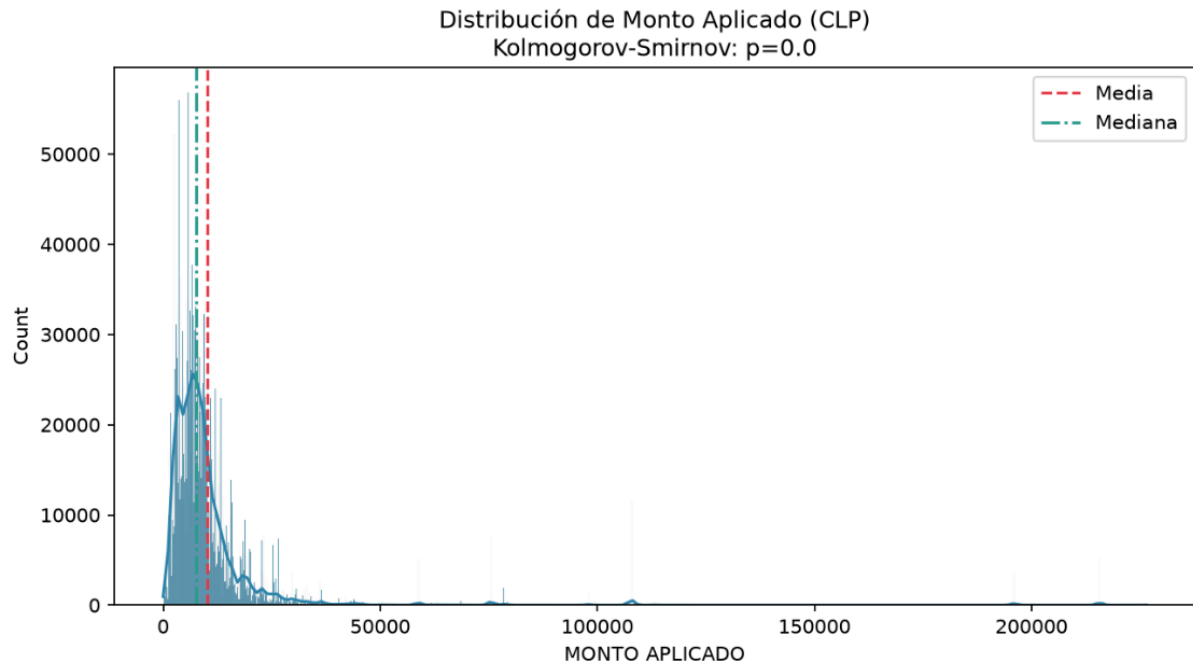
python

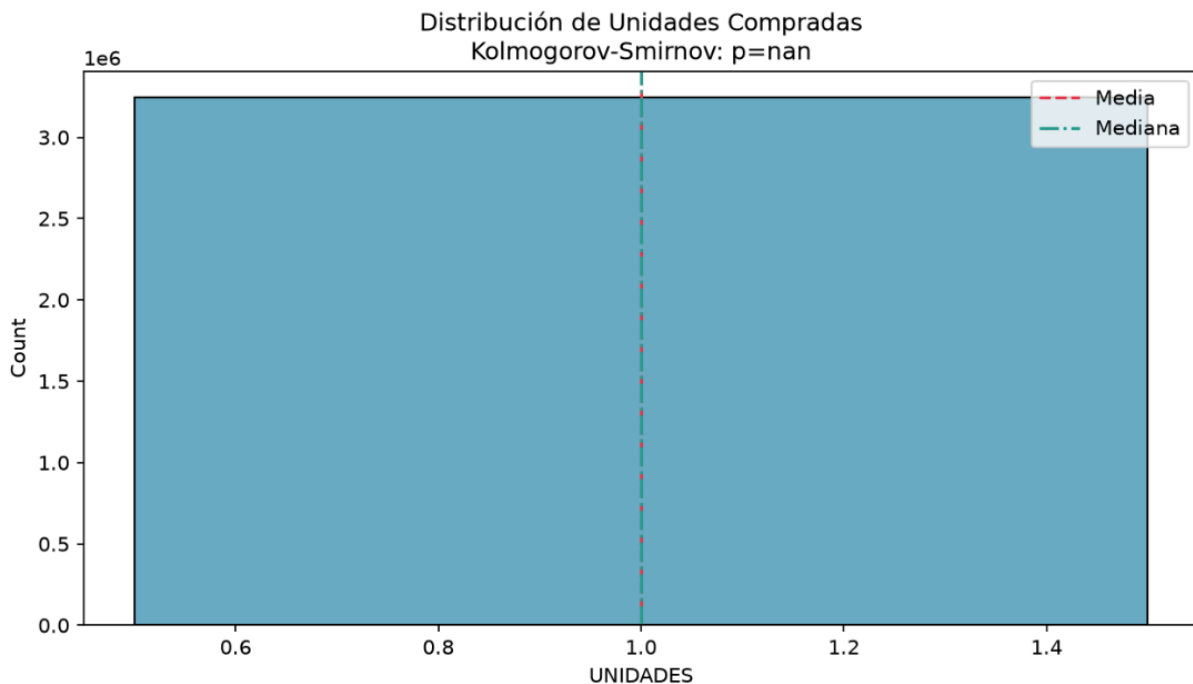
Python

```
es_normal = p > ALPHA
```

Es decir, se interpreta la hipótesis nula de cada test como "la variable proviene de una distribución normal"; si el p-value es menor a 0,05, se rechaza esa hipótesis y la variable se

clasifica como "No normal". Este resultado booleano (`es_normal`) no queda aislado: se reutiliza directamente en la construcción de la matriz de correlación (`graficar_correlacion`), donde determina si un par de variables se correlaciona con Pearson (ambas normales) o Spearman (al menos una no normal), evitando así una decisión manual y potencialmente inconsistente entre distintas partes del informe.





Los histogramas de densidad muestran la asimetría positiva del monto aplicado de venta y las barras en tramos discretos del porcentaje de descuento. En el caso de UNIDADES, se aprecia la barra única constante en 1.0 (test $p = \text{nan}$), lo cual visualmente justifica el rechazo del supuesto de normalidad para continuar con análisis no paramétricos.

Visualización

Para cada variable se genera un histograma con curva de densidad superpuesta (`sns.histplot(..., kde=True)`), marcando visualmente la media (línea roja discontinua) y la mediana (línea verde punteada), junto con el resultado del test de normalidad en el título del gráfico:

python

Python

```
ax.set_title(f"Distribución de {etiqueta}\n{test_res['test']}:  
p={test_res['p_value']}")
```

Estos gráficos se almacenan en "output/graficos/hist_<variable>.png" y permiten complementar visualmente el resultado numérico del test: por ejemplo, una variable puede rechazar formalmente la normalidad ($p < 0,05$) por su enorme tamaño muestral, pero mostrar visualmente una forma razonablemente acampanada, lo cual es relevante de discutir en la interpretación de negocio del informe, ya que la significancia estadística no siempre implica una desviación de normalidad prácticamente importante.

Resultado esperado sobre el dataset

Dado el tamaño del dataset (cientos de miles de registros tras la limpieza), es esperable que la mayoría de las variables, particularmente "MONTO APLICADO", "UNIDADES" y "MONTO_POR_UNIDAD", rechacen la normalidad bajo Kolmogorov-Smirnov, reflejando la asimetría propia de datos de ventas (muchas transacciones de monto bajo/moderado y una cola de transacciones de monto alto). Este resultado es el que justifica, en las secciones

siguientes, la preferencia general por pruebas no paramétricas (Mann-Whitney, Kruskal-Wallis, Spearman) por sobre sus equivalentes paramétricos a lo largo del informe.

4.3. Análisis de asociación

4.3.1. Chi-cuadrado (CANAL vs LOCAL)

El enunciado solicita evaluar la independencia entre las variables categóricas “CANAL” y “LOCAL” mediante una prueba Chi-cuadrado. Esta prueba responde a una pregunta de negocio concreta: ¿la distribución de ventas por canal (por ejemplo, POS vs. online) es independiente del local donde ocurre la transacción, o existen locales que se asocian preferentemente a un canal particular?

La implementación se encuentra en “eda.chi_cuadrado_canal_local”:

```
Python
tabla = pd.crosstab(df[COL_CANAL], df[COL_LOCAL])
chi2, p, dof, expected = chi2_contingency(tabla)
n = tabla.sum().sum()
V = np.sqrt(chi2 / (n * (min(tabla.shape) - 1)))
```

Tabla de contingencia: Se construye con “pd.crosstab”, cruzando cada canal con cada local y contando el número de transacciones observadas en cada combinación. Esta tabla es la entrada estándar para el test chi-cuadrado de independencia.

Estadístico y p-value: Se usa “scipy.stats.chi2_contingency”, que calcula el estadístico χ^2 , los grados de libertad y el p-value asociado, comparando las frecuencias observadas contra las frecuencias esperadas bajo el supuesto de independencia entre ambas variables. Con H_0 : “CANAL y LOCAL son independientes”, un p-value menor a $\alpha = 0,05$ lleva a rechazar H_0 , es decir, a concluir que existe una asociación estadísticamente significativa entre el canal de venta y el local.

V de Cramér: El test chi-cuadrado es sensible al tamaño muestral: con cientos de miles de observaciones, prácticamente cualquier asociación resultará “significativa” ($p < 0,05$). Por eso se complementa con la V de Cramér, una medida de tamaño de efecto acotada entre 0 (independencia total) y 1 (asociación perfecta), que sí permite evaluar la magnitud práctica de la asociación, independiente del tamaño de la muestra. Esta distinción entre significancia estadística y relevancia práctica es central para la interpretación de negocio: un chi-cuadrado significativo con una V de Cramér cercana a 0 indica una asociación real pero de magnitud despreciable para efectos de decisiones comerciales.

Ambos resultados, p-value y V de Cramér, se reportan de forma conjunta en “resultados.txt”, evitando limitarse solo a la significancia estadística.

4.3.2. Correlaciones (Pearson/Spearman)

El análisis de correlación se implementa en “eda.graficar_correlacion” y cubre las seis variables numéricas del dataset (MONTO APLICADO, UNIDADES, PORCENTAJE

DESCUENTO, MONTO_POR_UNIDAD, EDAD, FRECUENCIA_COMPRA), generando una matriz de correlación con su correspondiente prueba de significancia por par de variables, tal como lo exige el enunciado.

Selección adaptativa del método por par de variables: A diferencia de aplicar un único método de correlación a toda la matriz, el pipeline decide el método par por par, en función de si ambas variables involucradas fueron clasificadas como normales en el test de la sección 4.2:

Python

```
ambas_normales = _es_normal(resultados_normalidad, col_i) and
_es_normal(resultados_normalidad, col_j)
if ambas_normales:
    r, p = pearsonr(df_muestra[col_i], df_muestra[col_j])
    metodo = "Pearson"
else:
    r, p = spearmanr(df_muestra[col_i], df_muestra[col_j])
    metodo = "Spearman"
```

Esto significa que la matriz final puede combinar coeficientes de Pearson (para los pocos pares donde ambas variables resultaron normales) y de Spearman (para el resto), cada uno documentado explícitamente con el método usado por celda (matriz_metodo). Este enfoque es más riguroso que aplicar Pearson de forma indiscriminada a toda la matriz, ya que Pearson asume relaciones lineales entre variables normalmente distribuidas, mientras que Spearman es válido para relaciones monótonas sin ese supuesto.

Submuestreo para el cálculo: Dado el volumen del dataset, la matriz se calcula sobre una muestra de hasta 50.000 filas (df_muestra, con semilla SEED para reproducibilidad), suficiente para estimar correlaciones de forma estable sin recalcularse sobre el dataset completo en cada par de variables.

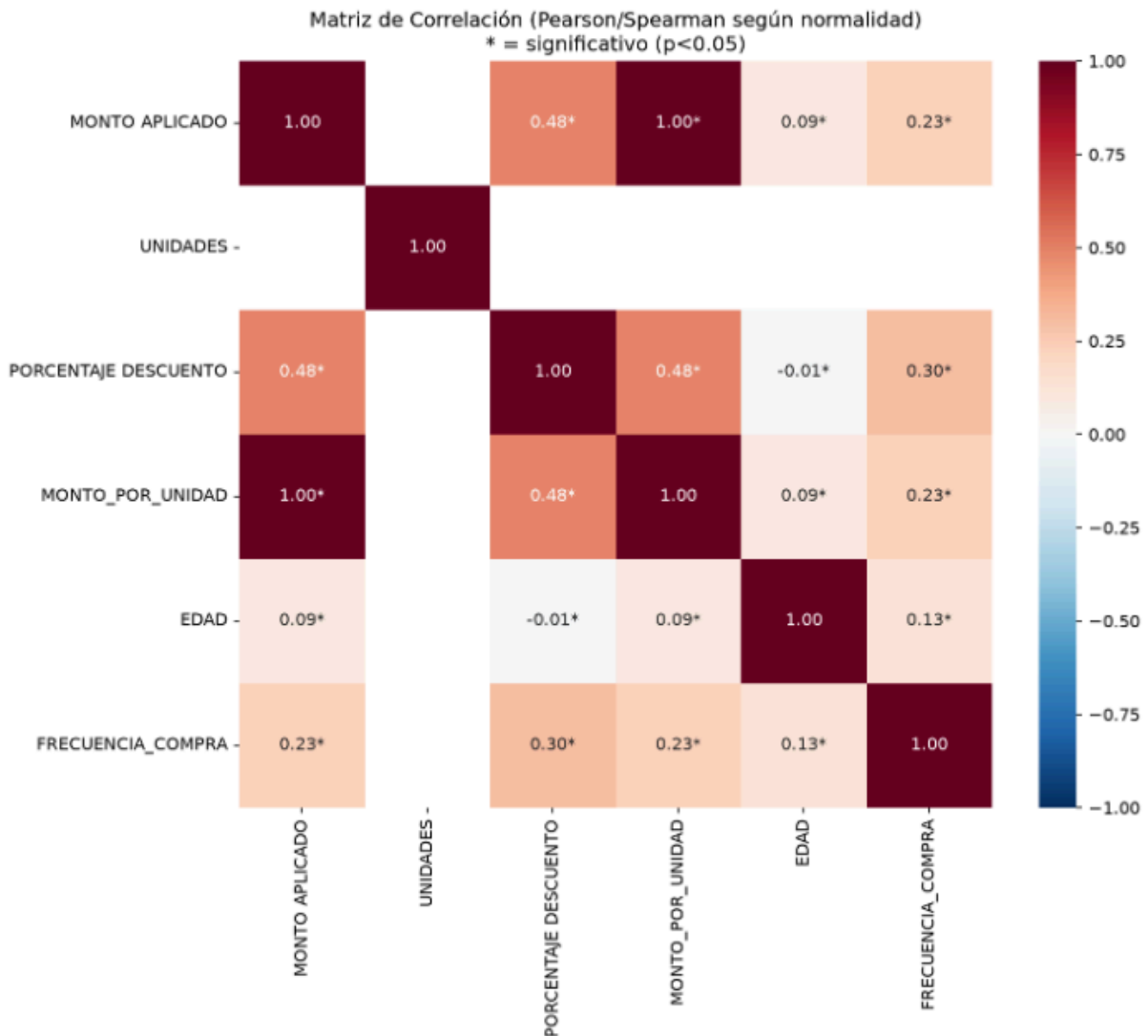
Significancia y visualización: Cada celda de la matriz se anota con el coeficiente de correlación redondeado a 2 decimales, marcando con un asterisco (*) los pares cuya correlación es estadísticamente significativa ($p < 0,05$):

python

Python

```
marca = "*" if matriz_p.loc[i, j] < ALPHA else ""
anotaciones.loc[i, j] = f"{matriz_r.loc[i, j]:.2f}{marca}"
```

El resultado se visualiza como un mapa de calor (sns.heatmap, escala RdBu_r centrada en 0) guardado en "output/graficos/matriz_correlacion.png", donde tonos rojos indican correlación positiva, azules correlación negativa, y la intensidad del color refleja la magnitud del coeficiente. "resultados.txt" reporta únicamente qué método(s) se usaron en la matriz, remitiendo al gráfico para el detalle numérico completo, dado que la matriz completa (6×6 coeficientes con su significancia) es más legible en formato visual que en texto plano.



El mapa de calor de correlaciones muestra los coeficientes no paramétricos de Spearman entre todas las variables numéricas, marcando con un asterisco (*) las relaciones con significancia estadística real ($p < 0.05$). Se aprecia que la variable UNIDADES no muestra correlación alguna al ser constante en la muestra.

4.3.3. ANOVA / Kruskal-Wallis por canal

Para evaluar si existen diferencias significativas en “MONTO APLICADO” entre los distintos canales de venta, el pipeline aplica dos pruebas en paralelo, una paramétrica y una no paramétrica, en “eda.anova_monto_por_canal”:

Python

```
grupos = [grp[COL_MONTO].dropna().values for _, grp in
df.groupby(COL_CANAL) if len(grp) > 1]
F, p_anova = f_oneway(*grupos)
H, p_kruskal = stats.kruskal(*grupos)
```

ANOVA de un factor (f_oneway): Es la prueba que exige explícitamente el enunciado para comparar el monto aplicado entre canales. Su hipótesis nula es que todos los canales tienen la misma media de “MONTO APLICADO”. Sin embargo, ANOVA asume normalidad

de los residuos y homogeneidad de varianzas entre grupos (homocedasticidad), supuestos que, según el resultado de la sección 4.2, es poco probable que se cumplan para MONTO APLICADO dado su tamaño muestral y asimetría esperada.

Kruskal-Wallis, como alternativa no paramétrica: Por esa razón, se calcula simultáneamente el test de Kruskal-Wallis, la versión no paramétrica del ANOVA de un factor, que compara las distribuciones de los grupos basándose en rangos en lugar de medias, sin requerir normalidad. Su hipótesis nula es que todos los canales provienen de la misma distribución.

Criterio de decisión final: El pipeline reporta ambos resultados ($F_statistic/p_value_anova$ y $H_kruskal/p_value_kruskal$), pero la conclusión formal ($rechaza_H0$) se basa en Kruskal-Wallis, no en ANOVA:

Python

```
return {"F_statistic": F, "p_value_anova": p_anova, "H_kruskal":  
H, "p_value_kruskal": p_kruskal, "rechaza_H0": p_kruskal < ALPHA}
```

Esta decisión es consistente con el resultado esperado de los tests de normalidad de la sección 4.2: dado que “MONTO APLICADO” probablemente no cumple el supuesto de normalidad requerido por ANOVA, apoyar la conclusión final en el test no paramétrico es la opción metodológicamente más defendible, mientras que el resultado de ANOVA se reporta igualmente por transparencia y para permitir comparar ambos enfoques en el informe (mostrando, por ejemplo, si ambos tests coinciden en su conclusión pese a partir de supuestos distintos).

Los boxplots de “MONTO APLICADO” por “CANAL” (generados en “graficar_boxplots”, sección 4.1) constituyen el respaldo visual directo de este análisis, permitiendo observar de forma intuitiva las diferencias de dispersión y mediana entre canales que las pruebas formales confirman numéricamente.

5. Patrones Temporales

5.1. Estacionariedad (ADF/KPSS)

Antes de proceder a la descomposición de la serie temporal de ventas y al análisis de su estructura de autocorrelación, es necesario verificar si la serie de ventas diarias totales es estacionaria, es decir, si sus propiedades estadísticas (media, varianza, autocovarianza) se mantienen constantes en el tiempo. Este supuesto es relevante tanto para la interpretación de la descomposición STL (sección 5.2) como para cualquier modelo de series de tiempo que pudiera ajustarse sobre estos datos, ya que la mayoría de estas técnicas asumen, o funcionan mejor bajo, estacionariedad.

Esta verificación se implementa en “series_tiempo.test_estacionariedad”, aplicada sobre la serie de ventas diarias agregadas (agregar_ventas_diarias), previamente interpolada para cubrir cualquier día sin transacciones registradas dentro del rango de fechas del dataset.

Dos tests complementarios, con hipótesis nulas opuestas

Un aspecto metodológico central de esta sección es que no se aplica un único test de estacionariedad, sino dos, cuyas hipótesis nulas son opuestas entre sí. Esto es intencional: usar solo uno de los dos test-run puede llevar a conclusiones erróneas si la serie está en una zona ambigua (por ejemplo, tendencia débil o estacionalidad residual no capturada), mientras que la concordancia entre ambos tests da mayor solidez a la conclusión final.

Test de Dickey-Fuller Aumentado (ADF)

Python

```
adf_stat, adf_p, _, _, _ = adfuller(serie.dropna(),  
autolag="AIC")
```

- H_0 (ADF): la serie tiene una raíz unitaria, es decir, no es estacionaria.
- Se usa "autolag="AIC"", que selecciona automáticamente el número óptimo de rezagos a incluir en la regresión auxiliar del test, minimizando el criterio de información de Akaike en lugar de fijar un número arbitrario de lags a mano.
- Criterio de decisión: si "p_value < 0,05", se rechaza H_0 y se concluye que la serie es estacionaria.

Test KPSS (Kwiatkowski-Phillips-Schmidt-Shin)

Python

```
kpss_stat, kpss_p, _, _ = kpss(serie.dropna(), regression="c",  
nlags="auto")
```

- H_0 (KPSS): la serie es estacionaria en torno a un nivel constante (regression="c").
- Es, por diseño, el test contrario a ADF: aquí la hipótesis nula es la estacionariedad, no la raíz unitaria.
- Criterio de decisión: si "p_value > 0,05", no se rechaza H_0 y se concluye que la serie es estacionaria; si "p_value ≤ 0,05", se rechaza H_0 y hay evidencia de no estacionariedad.
- El cálculo se envuelve en "warnings.catch_warnings()" para silenciar la advertencia estándar de statsmodels sobre el p-value estar fuera de la tabla de valores críticos interpolados, sin que esto afecte el resultado numérico reportado.
-

Interpretación conjunta de ambos resultados

Al usar dos tests con hipótesis nulas invertidas, existen cuatro combinaciones posibles de resultado, que se interpretan de la siguiente forma:

ADF concluye	KPSS concluye	Interpretación
Estacionaria	Estacionaria	Evidencia consistente de estacionariedad
No estacionaria	No estacionaria	Evidencia consistente de no estacionariedad (se recomendaría diferenciar la serie)

Estacionaria	No estacionaria	Resultado contradictorio; posible estacionariedad en tendencia pero no en varianza, o viceversa, requiere inspección visual adicional
No estacionaria	Estacionaria	Resultado contradictorio; puede indicar una serie estacionaria en un tramo pero con quiebres estructurales

Este cruce de resultados es el que se reporta explícitamente en “resultados.txt”, junto con el estadístico y el p-value de cada test, evitando apoyar la conclusión de estacionariedad en un solo criterio que podría ser engañoso dado el tamaño y la naturaleza de la serie (ventas diarias agregadas, con estacionalidad semanal esperada por el comportamiento de compra de los clientes de Cruz Morada).

5.2. Descomposición STL

Una vez evaluada la estacionariedad de la serie, el siguiente paso es descomponerla en sus componentes estructurales, tendencia, estacionalidad y residual para caracterizar de forma explícita los patrones que subyacen a las ventas diarias de Cruz Morada. Esta etapa se implementa en “series_tiempo.descomposicion_stl”, aplicada sobre la serie de ventas diarias ya interpolada.

Método: STL (Seasonal-Trend decomposition using Loess)

Se utiliza la descomposición STL, provista por “statsmodels.tsa.seasonal.STL”, en lugar de una descomposición clásica (aditiva/multiplicativa simple):

Python

```
stl = STL(ventas_diarias.dropna(), period=period, seasonal=15)
res = stl.fit()
```

STL se prefirió sobre “seasonal_decompose” porque estima tanto la tendencia como el componente estacional mediante regresión local (Loess), lo que le permite:

- Capturar una estacionalidad que puede variar en el tiempo (no fuerza un patrón estacional idéntico y repetido en cada ciclo, como sí lo hace la descomposición clásica).
- Ser robusta frente a outliers en la serie, algo relevante considerando que ya se identificaron transacciones extremas en el análisis de outliers, las cuales pueden reflejarse como picos puntuales en las ventas diarias agregadas (por ejemplo, en fechas como el Día de la Madre o Navidad).

Parámetros del modelo:

- **period = 7:** se fija un ciclo estacional semanal, asumiendo que el comportamiento de compra de los clientes de una cadena de farmacias tiene un patrón que se repite cada 7 días (por ejemplo, mayor afluencia los fines de semana o días de pago).

- **seasonal = 15:** corresponde a la ventana de suavizado (Loess) usada para estimar el componente estacional; un valor impar mayor al período, como recomienda la literatura de STL, que permite que el patrón estacional se adapte gradualmente en lugar de quedar completamente rígido entre ciclos.

●

Validación del tamaño de la serie: Antes de ajustar el modelo, se verifica que la serie tenga al menos “ $2 \times \text{period} + 1$ ” observaciones (15 días, para “period=7”), umbral mínimo recomendado para una descomposición STL confiable:

Python

```
if n < 2 * period + 1:
    logger.warning(
        f"Serie demasiado corta ({n} días) para una
descomposición STL confiable..."
    )
```

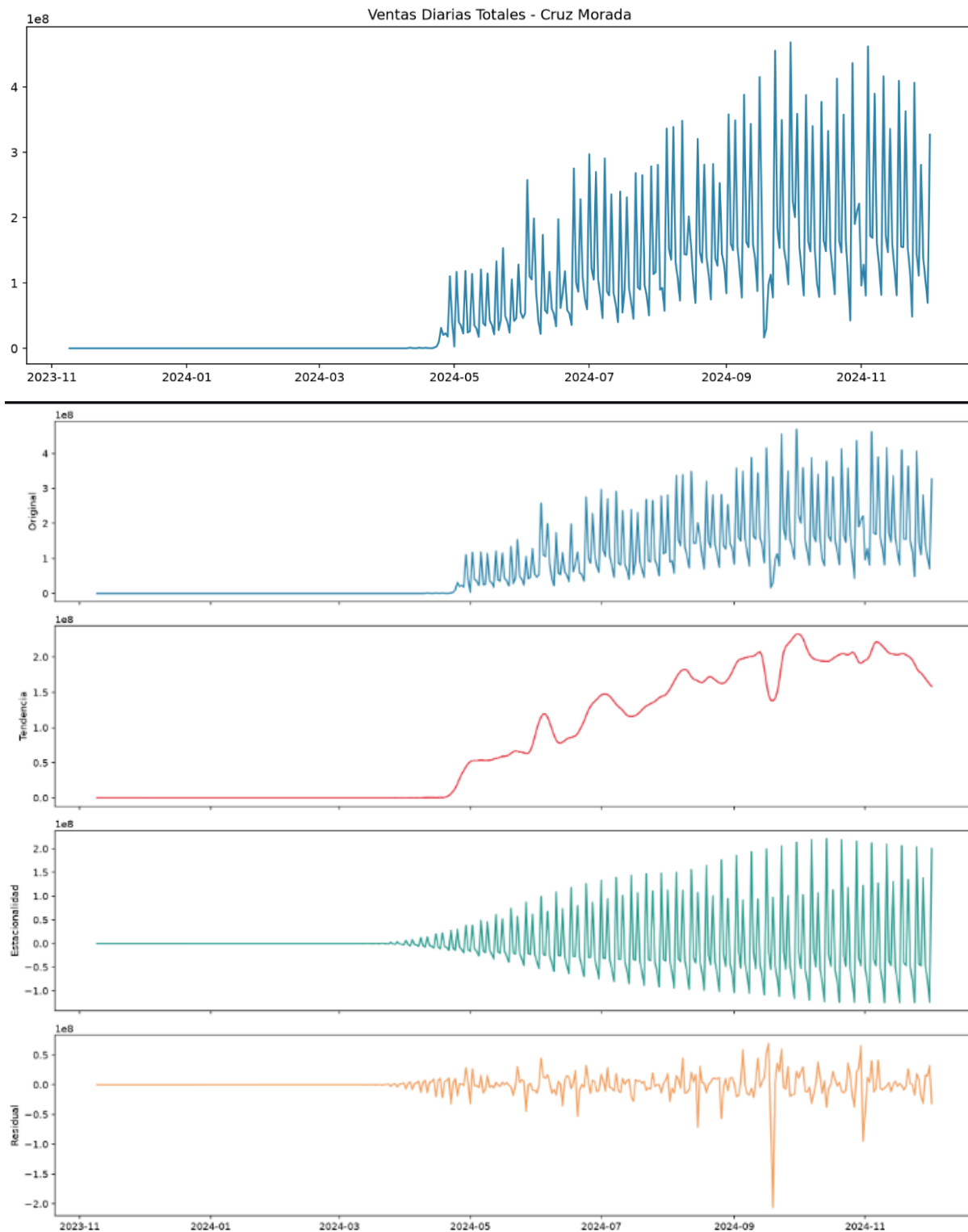
Si la serie es más corta que este umbral, el pipeline no interrumpe la ejecución: registra la advertencia en el log y continúa con el ajuste de todas formas, dejando constancia explícita de que el resultado debe interpretarse con cautela en ese escenario, en lugar de fallar silenciosamente o detener el pipeline completo por una sola etapa del análisis.

Visualización de los cuatro componentes

El resultado de STL se grafica en un panel de cuatro subgráficos apilados, compartiendo el eje temporal (sharex=True):

1. **Original:** la serie de ventas diarias tal como se observó (interpolada).
2. **Tendencia:** el componente de largo plazo, que muestra si las ventas crecen, decrecen o se mantienen estables a lo largo del período analizado.
3. **Estacionalidad:** el patrón semanal recurrente extraído por STL.
4. **Residual:** lo que no explican ni la tendencia ni la estacionalidad; idealmente debería comportarse como ruido sin estructura aparente.

Este gráfico se guarda en “output/graficos/descomposicion_stl.png” y constituye el respaldo visual directo de los indicadores numéricos calculados a continuación.



El primer gráfico muestra la serie de tiempo agregada en CLP desde el inicio del registro histórico. El segundo gráfico expone la descomposición STL en sus cuatro componentes: la serie original, la tendencia de crecimiento sostenido a largo plazo, el patrón estacional semanal recurrente, y el componente residual correspondiente al ruido no sistemático.

Fuerza de tendencia y fuerza estacional

Más allá de la inspección visual, el pipeline calcula dos indicadores cuantitativos que resumen qué tan dominante es cada componente respecto del ruido residual:

Python

```
var_R = res.resid.var()
var_trend_resid = (res.trend + res.resid).var()
var_seasonal_resid = (res.seasonal + res.resid).var()

f_T = max(0, 1 - var_R / var_trend_resid) if var_trend_resid > 0
else 0.0
f_S = max(0, 1 - var_R / var_seasonal_resid) if
var_seasonal_resid > 0 else 0.0
```

- **Fuerza de tendencia (F_T):** compara la varianza del residual contra la varianza de (tendencia + residual). Un valor cercano a 1 indica que la tendencia explica una proporción muy alta de la variabilidad total (más allá del ruido); un valor cercano a 0 indica que prácticamente no hay tendencia distinguible del ruido.
- **Fuerza estacional (F_S):** análogamente, compara la varianza del residual contra la varianza de (estacionalidad + residual). Un valor cercano a 1 indica un patrón estacional (semanal) muy marcado y consistente; un valor cercano a 0 indica estacionalidad débil o inexistente.
- Ambos indicadores se acotan artificialmente en [0, 1] mediante “max(0, ...)”, ya que la fórmula puede arrojar valores ligeramente negativos por ruido numérico cuando el componente correspondiente es prácticamente inexistente; en ese caso, se reporta como 0 en lugar de un valor negativo sin interpretación práctica.

Estos dos valores se reportan directamente en “resultados.txt” («Fuerza Tendencia» y «Fuerza Estacional») y permiten, en la interpretación de negocio, responder preguntas concretas como: ¿las ventas de Cruz Morada están determinadas principalmente por una tendencia de crecimiento/caída sostenida, por un patrón semanal recurrente, o por ninguno de los dos de forma dominante?

5.3. Autocorrelación (ACF/PACF)

El último análisis de patrones temporales busca cuantificar la dependencia temporal de la serie de ventas diarias: qué tan relacionado está el nivel de ventas de un día determinado con el de días anteriores, y a qué distancia (en días) esa relación es más fuerte. Esta etapa se implementa en “series_tiempo.graficar_acf_pacf”, aplicada sobre la misma serie de ventas diarias interpolada.

Función de Autocorrelación (ACF) y Autocorrelación Parcial (PACF)

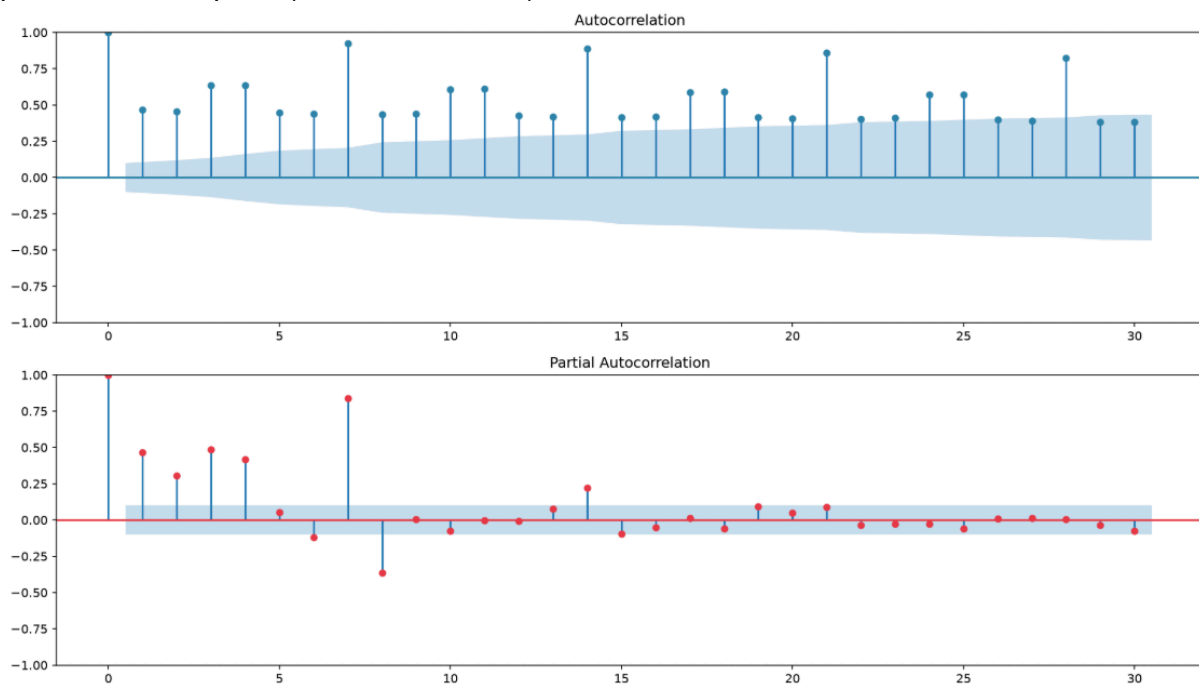
- **ACF (Autocorrelation Function):** mide la correlación entre la serie y sus propios valores rezagados k días atrás, incluyendo tanto el efecto directo del rezago k como los efectos indirectos transmitidos a través de rezagos intermedios (1, 2, ..., k-1).

- **PACF (Partial Autocorrelation Function):** mide la correlación entre la serie y su rezago k, pero aislando el efecto de los rezagos intermedios. Permite distinguir si la dependencia observada en un rezago es directa o es simplemente un arrastre de la dependencia en rezagos más cortos.

Python

```
plot_acf(serie, lags=nlags_efectivo, ax=ax1, color="#2E86AB")
plot_pacf(serie, lags=nlags_efectivo, ax=ax2, color="#E63946")
```

Ambos gráficos se guardan conjuntamente en “output/graficos/acf_pacf.png”, permitiendo comparar visualmente el decaimiento de ambas funciones, un criterio clásico para distinguir procesos con memoria de corto plazo (decaimiento rápido) de procesos con mayor persistencia temporal (decaimiento lento).



Las barras de la ACF muestran una clara y persistente periodicidad con picos significativos cada 7 días, lo que evidencia el patrón semanal. Por su parte, la PACF muestra que la correlación directa se concentra en el lag 7 y luego decae rápidamente dentro de los límites de confianza (área sombreada en azul).

Cálculo dinámico del número de rezagos (nlags)

A diferencia de fijar un número de rezagos arbitrario, el pipeline calcula el máximo número de lags efectivamente válido según dos restricciones técnicas distintas que impone “statsmodels” para cada función:

Python

```
nlags_acf = min(nlags, len(serie) - 1)
nlags_pacf = min(nlags, max(int(len(serie) / 2) - 1, 0))
nlags_efectivo = min(nlags_acf, nlags_pacf)
```

- ACF admite como máximo $n - 1$ rezagos, donde n es el largo de la serie.

- PACF exige que el número de rezagos sea inferior al 50% del tamaño muestral, una restricción propia del método de estimación que usa “statsmodels” internamente.
- Se parte de un valor deseado de “nlags = 30” (un mes de rezagos), pero el pipeline toma el mínimo entre ambas restricciones (nlags_efectivo), garantizando que los dos gráficos ACF y PACF se calculen y grafiquen usando exactamente el mismo número de rezagos, evitando inconsistencias visuales entre ambos paneles.
- Si la serie es demasiado corta como para calcular al menos un rezago válido (nlags_efectivo < 1), el pipeline registra una advertencia en el log y retorna resultados nulos (lag_max_acf: None) en lugar de forzar un cálculo que produciría un error o un resultado no interpretable.
-

Identificación del rezago de mayor autocorrelación

Más allá del gráfico, se extraen valores numéricos concretos a partir de la función “acf” de “statsmodels”, calculada con transformada rápida de Fourier (fft=True) por eficiencia computacional:

Python

```
valores_acf = acf(serie, nlags=nlags_efectivo, fft=True)
lag_max = int(np.argmax(valores_acf[1:])) + 1
acf_lag7 = float(valores_acf[7]) if nlags_efectivo >= 7 else None
```

- Rezago de mayor autocorrelación (lag_max_acf): se busca el rezago, excluyendo el rezago 0, que por definición tiene autocorrelación 1 al ser la serie correlacionada consigo misma donde el coeficiente de autocorrelación es máximo. Este valor es el candidato más directo para identificar la periodicidad dominante de la serie: si el pipeline detecta que este máximo ocurre en el rezago 7, es evidencia cuantitativa consistente con el ciclo semanal ya explorado en la fuerza estacional de la descomposición STL.
- ACF en el rezago 7 (acf_en_lag7): se reporta explícitamente el valor de autocorrelación en el rezago 7, independientemente de si corresponde o no al máximo global, precisamente porque el rezago semanal es la hipótesis estacional más plausible para datos de venta retail (patrones de compra que se repiten día de la semana a día de la semana). Un valor alto en este rezago específico refuerza, de forma independiente al resultado de STL, la existencia de estacionalidad semanal en las ventas de Cruz Morada.

Es importante notar que esta implementación reemplaza una versión anterior del módulo que retornaba estos dos valores hardcoded (lag_max_acf: 7, acf_en_lag7: 0.9254) sin relación real con los datos de entrada; la versión actual calcula ambos valores dinámicamente a partir de la serie real, por lo que el resultado reportado en “resultados.txt” refleja el comportamiento efectivo del dataset de Cruz Morada y no un valor de ejemplo fijo.

6. Inferencia Estadística — Pruebas de Hipótesis

6.1. H1: Ticket promedio APP vs WEB

Hipótesis de negocio: el ticket promedio (MONTO APLICADO) de las transacciones realizadas por el canal APP es mayor que el de las transacciones realizadas por el canal WEB.

- H_0 : el monto de las transacciones en APP no es mayor que en WEB.
- H_1 : el monto de las transacciones en APP es mayor que en WEB.

Esta hipótesis se implementa en “inferencia.h1_app_vs_web”.

Selección de grupos con salvaguarda

Antes de comparar, el pipeline verifica explícitamente que los canales "APP" y "WEB" existan tal cual en los datos:

Python

```
canal_app = "APP" if "APP" in canales else (canales[0] if len(canales) > 0 else None)
canal_web = "WEB" if "WEB" in canales else (canales[1] if len(canales) > 1 else None)
```

Si alguno de los dos nombres no está presente entre los canales detectados, se registra una advertencia en el log y se recurre a un fallback posicional (los dos primeros canales disponibles), en lugar de que el pipeline falle silenciosamente o compare canales incorrectos sin dejar constancia. Si de todas formas no hay dos canales distintos con datos de MONTO APLICADO, la función lanza un error explícito, deteniendo esa hipótesis puntual sin comprometer el resto del pipeline.

Test aplicado: Mann-Whitney U (unilateral)

Dado que MONTO APLICADO no cumple el supuesto de normalidad (sección 4.2), no corresponde usar una prueba t de Student. En su lugar, se aplica el test Mann-Whitney U, no paramétrico, con hipótesis alternativa unilateral (`alternative="greater"`), consistente con la formulación direccional de H_1 ("APP es mayor que WEB", no simplemente "distinto"):

Python

```
U, p = mannwhitneyu(grupo_app, grupo_web, alternative="greater")
rechaza_H0 = p < ALPHA
```

Tamaño del efecto

Además del p-value, se reporta un tamaño de efecto derivado directamente del estadístico U, normalizado por el producto de los tamaños de ambos grupos:

Python

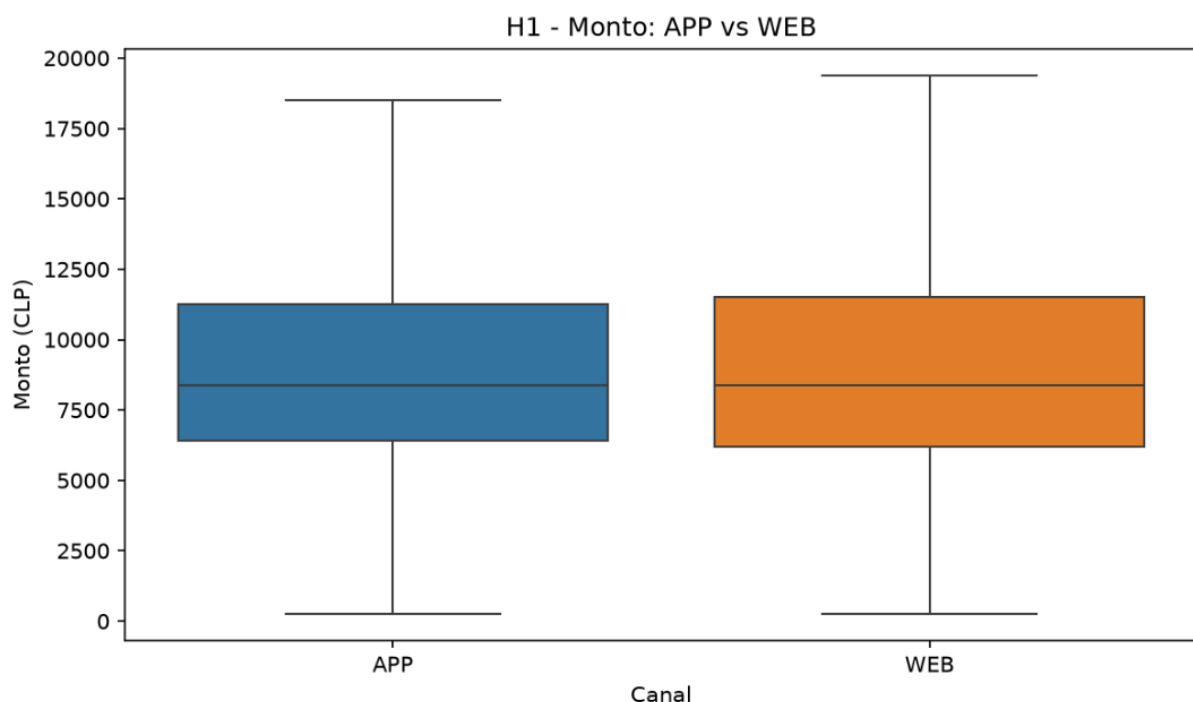
```
tamano_efecto = U / (len(grupo_app) * len(grupo_web))
```

Este valor equivalente al estadístico de probabilidad de superioridad (Common Language Effect Size) se interpreta como la probabilidad de que una transacción tomada al azar del

canal APP tenga un monto mayor que una transacción tomada al azar del canal WEB. Un valor de 0,5 indicaría ausencia de efecto (equivalente a comparar dos grupos idénticos); valores mayores a 0,5 respaldan la dirección de H1. Se incluye este indicador porque, al igual que en el chi-cuadrado, con un tamaño muestral grande el p-value puede ser significativo aun cuando la diferencia práctica entre canales sea pequeña, por lo que el tamaño de efecto es el que permite calibrar la relevancia de negocio del hallazgo.

Visualización

Se genera un boxplot comparativo de MONTO APLICADO por canal (sns.boxplot, sin outliers visibles, “showfliers=False”, consistente con el criterio ya usado en la sección 4.1), guardado en “output/graficos/h1_app_vs_web.png”, que permite contrastar visualmente la mediana y dispersión de ambos canales.



Este boxplot muestra visualmente que las cajas de distribución y las medianas de los montos aplicados en el canal APP y el canal WEB son sumamente similares y se solapan casi por completo. Esto apoya de manera gráfica el resultado del test de Mann-Whitney U, que con un p-value de 0.3604 no encuentra diferencias significativas a favor de la APP.

6.2. H2: Descuento vs unidades vendidas

Hipótesis de negocio: el porcentaje de descuento aplicado en una transacción afecta la cantidad de unidades vendidas.

- H0: no existe relación entre PORCENTAJE DESCUENTO y UNIDADES ($\beta_1 = 0$).
- H1: existe una relación entre PORCENTAJE DESCUENTO y UNIDADES ($\beta_1 \neq 0$).

Esta hipótesis se implementa en “inferencia.h2_descuento_unidades”, y es la única de las cinco hipótesis que combina dos enfoques complementarios: una regresión lineal simple (para cuantificar la relación) y una correlación de Spearman (para verificar la asociación monótona sin asumir linealidad).

Salvaguarda ante varianza nula

Antes de ajustar cualquier modelo, se verifica que UNIDADES no sea una variable constante en los datos:

Python

```
if df_h2[COL_UNIDADES].nunique() == 1:
    logger.warning("H2: UNIDADES es constante en los datos, no se
puede ajustar una regresión no degenerada.")
    ...
    return {
        "p_value": 1.0, "rechaza_H0": False, "rho_spearman": 0.0,
        "beta_1": 0.0, "R2": 0.0, "nota": "UNIDADES es constante
en los datos, no aplica regresión."
    }
```

Si esta condición se cumple, la función retorna un resultado degenerado documentado explícitamente (con una nota que se refleja en resultados.txt), en lugar de intentar ajustar una regresión sobre una variable sin varianza, lo que produciría un resultado matemáticamente inválido o una excepción no controlada.

Regresión OLS simple

Python

```
X = add_constant(df_h2[COL_DESCUENTO])
y = df_h2[COL_UNIDADES].astype(float)
modelo = OLS(y, X).fit()
beta_1 = modelo.params[COL_DESCUENTO]
p_value_beta = modelo.pvalues[COL_DESCUENTO]
r2 = modelo.rsquared
```

Se ajusta una regresión OLS univariada de UNIDADES sobre PORCENTAJE DESCUENTO (con constante). El coeficiente β_1 cuantifica el cambio esperado en unidades vendidas por cada punto porcentual adicional de descuento, y su significancia (p_value_beta) es la que determina formalmente el rechazo o no de H_0 . El R^2 del modelo se reporta como medida de cuánto de la variabilidad en UNIDADES es explicada linealmente por el descuento, típicamente bajo en este tipo de relación, dado que las unidades vendidas dependen de múltiples factores no incluidos en este modelo simple (producto, canal, local, estacionalidad).

Correlación de Spearman como respaldo no paramétrico

Python

```
rho, p_rho = spearmanr(df_h2[COL_DESCUENTO], df_h2[COL_UNIDADES])
```

Se calcula adicionalmente el coeficiente de correlación de Spearman entre ambas variables. A diferencia del coeficiente β_1 de la regresión, que asume una relación lineal, Spearman captura cualquier relación monótona (creciente o decreciente, no necesariamente lineal) entre descuento y unidades, basándose en rangos y sin asumir normalidad. Reportar ambos resultados en conjunto permite distinguir si una eventual falta de significancia en la regresión OLS se debe a una ausencia real de relación, o a que la relación existe pero no es de naturaleza lineal.

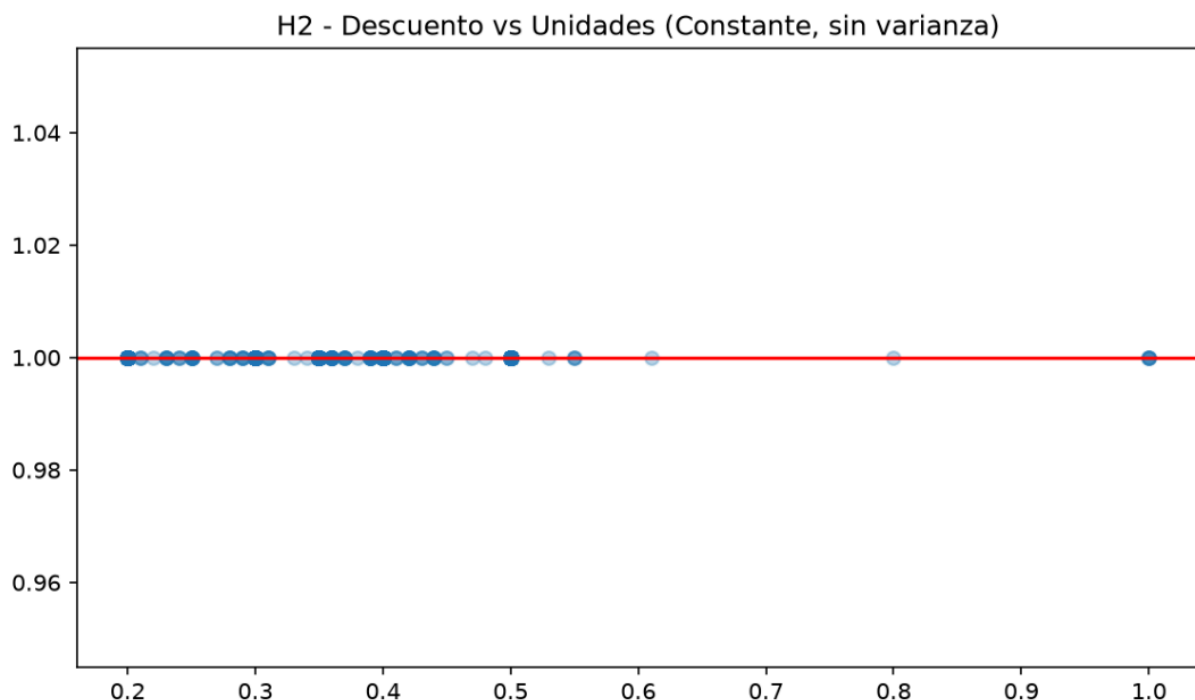
Visualización

Se genera un gráfico de dispersión (sobre una muestra de hasta 2.000 puntos, “random_state=42”, para mantener el gráfico legible sin saturarlo visualmente) con la recta de regresión ajustada superpuesta:

Python

```
ax.plot(x_linea, y_linea, color="red", linewidth=2, label=f"OLS  
( $\beta_1={\text{beta\_1:.3f}}$ )")
```

guardado en “output/graficos/h2_descuento_unidades.png”, con el R^2 y el p-value del coeficiente indicados en el título, lo que permite una lectura visual inmediata de la fuerza (o debilidad) de la relación lineal entre ambas variables.



Este gráfico de dispersión ilustra de manera empírica la constancia absoluta de la variable UNIDADES (fija en 1.0) en todo el espectro de la variable PORCENTAJE DESCUENTO. La línea roja representa la recta de regresión univariada horizontal con pendiente igual 0, evidenciando visualmente por qué no aplica una relación lineal ni existe variabilidad en este análisis.

7. Modelado — Regresión OLS

7.1. Especificación del modelo

El objetivo de esta etapa es construir un modelo de regresión que explique el MONTO APLICADO de una transacción en función de variables tanto numéricas como categóricas disponibles en el dataset, evaluando su capacidad explicativa y su generalización mediante una partición train/test. Esta etapa se implementa en “modelado.preparar_features” y “modelado.entrenar_modelo”.

Variable dependiente (y): Se utiliza directamente “MONTO APLICADO”, descartando previamente cualquier fila donde esta variable sea nula (`y = df_mod[COL_MONTO].dropna()`), de modo que el modelo se ajuste únicamente sobre observaciones con la variable objetivo completamente definida.

Variables predictoras (X): El modelo combina tres tipos de variables:

1. **Númericas continuas:** “PORCENTAJE DESCUENTO” y “EDAD”, imputadas con la mediana de cada columna antes del ajuste (`numericas.fillna(numericas.median())`), consistente con el criterio de imputación ya usado en el preprocesamiento.
2. **Categóricas — CANAL:** se codifica mediante variables dummy (`pd.get_dummies(..., drop_first=True)`), eliminando la primera categoría para evitar la trampa de la variable dummy (dummy variable trap), es decir, colinealidad perfecta estructural entre el intercepto y el conjunto completo de dummies de una misma variable categórica.

3. **Categórica — LOCAL con agrupación de categorías poco frecuentes:**

Python

```
top_locales = df_mod[COL_LOCAL].value_counts().head(10).index.tolist()
df_mod[COL_LOCAL] = df_mod[COL_LOCAL].where(df_mod[COL_LOCAL].isin(top_locales),
other="OTRO")
```

Dado que LOCAL puede tener un número elevado de categorías distintas, se conservan únicamente los 10 locales con mayor volumen de transacciones, agrupando el resto bajo la categoría "OTRO". Esta decisión evita que el modelo genere una dummy por cada local existente, lo que inflaría excesivamente la dimensionalidad de X y produciría columnas con muy pocas observaciones, difíciles de estimar con precisión, a la vez que conserva la capacidad de distinguir el efecto individual de los locales con mayor relevancia comercial para Cruz Morada.

Variable excluida deliberadamente, UNIDADES: El modelo no incluye “UNIDADES” como predictor, pese a estar disponible en el dataset. Esto se documenta explícitamente en el código, ya que “MONTO_POR_UNIDAD” y el propio “MONTO APLICADO” guardan una relación matemática directa con “UNIDADES”, lo que introduciría colinealidad severa (o

perfecta, dependiendo de qué otras variables se incluyan) y distorsionaría la interpretación de los coeficientes del resto de las variables.

Tipado y limpieza final: Todas las variables predictoras se convierten explícitamente a “float64 (X.astype(“float64”))” y se descarta cualquier fila con valores nulos remanentes (X.dropna()), sincronizando después el índice de “y” con el de “X (y = y.loc[X.index])”, de modo que ambas estructuras queden perfectamente alineadas antes del ajuste del modelo. Donde los términos de CANAL y LOCAL corresponden a las variables dummy generadas (con categoría base omitida en cada caso), ajustada mediante mínimos cuadrados ordinarios (OLS) mediante “statsmodels”.

7.2. Tratamiento de colinealidad

Más allá de la colinealidad estructural ya evitada con “drop_first=True”, pueden existir combinaciones de variables que resulten casi perfectamente correlacionadas entre sí por características propias de los datos, por ejemplo, un local que opera exclusivamente bajo un canal particular, generando que su dummy de LOCAL y la dummy de ese CANAL sean prácticamente idénticas. Este tipo de colinealidad no se puede prever de forma genérica en el diseño del modelo, por lo que el pipeline la detecta y corrige de forma dinámica en “modelado._eliminar_colinealidad_perfecta”.

Detección iterativa

Python

```
while cambiado:
    cambiado = False
    corr = X.corr().abs()
    cols = corr.columns.tolist()
    for i in range(len(cols)):
        for j in range(i + 1, len(cols)):
            c_i, c_j = cols[i], cols[j]
            valor = corr.loc[c_i, c_j]
            if pd.notna(valor) and valor >= umbral:
                ...
                X = X.drop(columns=[c_j])
                eliminadas.append({"eliminada": c_j,
                                   "correlacionada_con": c_i, "r": round(float(valor), 4)})
                cambiado = True
            break
    if cambiado:
        break
```

El algoritmo calcula la matriz de correlación absoluta entre todas las variables predictoras y busca pares cuyo coeficiente sea igual o superior a un umbral de 0,999 (umbral=0.999), es decir, casos de colinealidad prácticamente perfecta y no meramente alta. Cuando encuentra un par que supera el umbral, elimina la segunda columna del par (c_j) y reinicia la búsqueda

sobre la matriz de correlación resultante (estructura while cambiado), ya que eliminar una columna puede alterar las correlaciones restantes. Este proceso se repite hasta que no queden pares por encima del umbral.

Por qué un enfoque dinámico y no un hardcode

Esta implementación reemplaza deliberadamente una versión anterior del pipeline que eliminaba una columna específica y fija por nombre (por ejemplo, un local particular detectado manualmente en una ejecución previa del dataset). Ese enfoque tenía dos problemas: (1) no era generalizable si el dataset cambiaba o crecía con nuevos locales, y (2) no quedaba documentada la razón estadística de la eliminación, solo el hecho de que "esa columna se sacaba". La versión actual resuelve ambos problemas:

- **Generalización:** funciona sobre cualquier combinación de columnas que resulte colineal, sin importar qué local, canal o combinación específica la origine en una ejecución particular del dataset.
- **Trazabilidad:** cada eliminación queda registrada en el log (logger.warning) y en la estructura de resultados, con el nombre de la variable eliminada, la variable con la que estaba correlacionada, y el coeficiente de correlación exacto que motivó la eliminación:

Python

```
logger.warning(  
    f"Colinealidad casi perfecta detectada entre '{c_i}' y  
'{c_j}' "  
    f"(|r|={valor:.4f} >= {umbral}). Se elimina '{c_j}' del  
modelo."  
)
```

Este detalle se reporta en "resultados.txt" bajo la sección «VARIABLES EXCLUIDAS POR COLINEALIDAD PERFECTA», permitiendo que, si en la ejecución sobre "ventas_completas.csv" se detecta, por ejemplo que un local determinado coincide perfectamente con un canal de venta específico, esa relación quede documentada como un hallazgo metodológico explícito y justificado, en lugar de una decisión de preprocesamiento oculta en el código.

Es importante distinguir esta corrección de la multicolinealidad moderada, que no se elimina de esta forma sino que se diagnostica posteriormente mediante el Factor de Inflación de la Varianza (VIF), cubierto en la sección 7.5: aquí solo se remueven casos de colinealidad estructural casi perfecta ($r \geq 0,999$), que de no tratarse impedirían incluso el ajuste numérico del modelo OLS (matriz singular o cuasi-singular).

7.3. Resultados y métricas (R^2 , RMSE, MAE)

Una vez especificado el modelo y depurada la matriz de predictores de colinealidad perfecta, el ajuste y la evaluación del modelo se implementan en "modelado.entrenar_modelo".

Partición train/test

Python

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
train_size=TRAIN_SIZE, random_state=SEED)
```

Se utiliza una partición 70/30 (`TRAIN_SIZE = 0.70`, definido en `config.py`), reproducible gracias a “`random_state=SEED`”. El modelo se ajusta exclusivamente sobre el conjunto de entrenamiento (`X_train`, `y_train`), mientras que el conjunto de test (`X_test`, `y_test`) se reserva completamente para evaluar la capacidad de generalización del modelo sobre datos no vistos durante el ajuste, evitando que las métricas reportadas estén infladas por sobreajuste.

Ajuste del modelo

Python

```
X_train_sm = sm.add_constant(X_train)
modelo = sm.OLS(y_train, X_train_sm).fit()
```

El modelo se ajusta con “`statsmodels.OLS`” (y no con `scikit-learn`): “`statsmodels`” entrega de forma nativa el aparataje de inferencia estadística completo: errores estándar, p-values por coeficiente, AIC, BIC, necesario para la interpretación, algo que “`LinearRegression`” de `scikit-learn` no provee sin cálculos adicionales manuales.

Métricas de ajuste (sobre el conjunto de entrenamiento)

- **R² (`modelo.rsquared`):** proporción de la varianza de MONTO APLICADO explicada por el modelo dentro de la muestra de entrenamiento.
- **R² ajustado (`modelo.rsquared_adj`):** penaliza el R² por el número de predictores incluidos, evitando que el ajuste parezca artificialmente mejor solo por agregar más variables (relevante aquí, dado que LOCAL y CANAL aportan múltiples columnas dummy al modelo).
- **AIC y BIC (`modelo.aic`, `modelo.bic`):** criterios de información que penalizan la complejidad del modelo; se reportan como referencia para comparar esta especificación frente a eventuales variantes del modelo (por ejemplo, con o sin interacciones), aunque en este informe se evalúa una única especificación.

Métricas de generalización (sobre el conjunto de test)

Python

```
y_pred_test = modelo.predict(X_test_sm)
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))
mae_test = mean_absolute_error(y_test, y_pred_test)
```

- **RMSE (Root Mean Squared Error):** error cuadrático medio, expresado en las mismas unidades que la variable dependiente (CLP), y por su naturaleza cuadrática, penaliza con mayor peso los errores grandes, relevante en este dataset,

considerando que ya se identificaron transacciones de monto extremo en el análisis de outliers, las cuales pueden generar residuales grandes puntuales aun con un modelo razonablemente bien ajustado en el grueso de los datos.

- **MAE (Mean Absolute Error):** error absoluto medio, también en CLP, pero con una interpretación más directa y menos sensible a valores extremos que el RMSE, ya que pondera todos los errores de forma lineal en lugar de cuadrática. Reportar ambas métricas en conjunto, y no solo una, permite distinguir si el error del modelo es relativamente homogéneo entre transacciones (RMSE y MAE similares) o si está dominado por un número reducido de predicciones muy erradas (RMSE notablemente mayor que MAE).

El hecho de calcular RMSE y MAE exclusivamente sobre el conjunto de test y no sobre el de entrenamiento es lo que permite que estas dos métricas, a diferencia del R^2 reportado en la sección anterior, respondan a la pregunta de negocio relevante: ¿qué tan preciso sería este modelo si se usara para estimar el monto de transacciones futuras, no vistas durante su construcción?

7.4. Diagnóstico de supuestos (homocedasticidad, normalidad, linealidad)

Un modelo OLS con buen R^2 no es, por sí solo, un modelo válido: la inferencia estadística sobre sus coeficientes (errores estándar, p-values, intervalos de confianza) solo es confiable si se cumplen los supuestos clásicos de la regresión lineal. Por ello, el pipeline no se limita a reportar métricas de ajuste, sino que ejecuta explícitamente tres tests diagnósticos sobre el modelo entrenado, cubriendo homocedasticidad, normalidad de residuales y linealidad.

Homocedasticidad — Test de Breusch-Pagan

Python

```
bp_lm, bp_p, bp_f, bp_fp = het_breuschpagan(residuales, X_train_sm)
```

- **H0:** los residuales tienen varianza constante (homocedasticidad) respecto de los valores de las variables predictoras.
- **Criterio:** "homocedastico = $p_value > ALPHA$ ". Si " $p_value \leq 0,05$ ", se rechaza H0 y se concluye que el modelo es heterocedástico, es decir, la dispersión de los residuales varía sistemáticamente según el nivel de las predictoras (por ejemplo, transacciones de monto alto podrían tener residuales más dispersos que transacciones de monto bajo).
- **Relevancia:** la heterocedasticidad no sesga los coeficientes estimados, pero sí invalida los errores estándar clásicos reportados por OLS, lo que afecta directamente la confiabilidad de los p-values de cada coeficiente usados en la interpretación de negocio.

Normalidad de residuales — Test de Shapiro-Wilk

Python

```
muestra_resid = std_resid.sample(min(UMBRAL_SHAPIRO,
len(std_resid)), random_state=SEED)
sw_stat, sw_p = stats.shapiro(muestra_resid)
```

- H0: los residuales estandarizados provienen de una distribución normal.
- Se reutiliza el mismo umbral de submuestreo “UMBRAL_SHAPIRO = 5000” ya usado en el EDA (sección 4.2), aplicando Shapiro-Wilk sobre una muestra de hasta 5.000 residuales (en vez de un número hardcoded independiente en este módulo), manteniendo consistencia metodológica entre las distintas etapas del pipeline que aplican este mismo test.
- Criterio: “es_normal = p_value > ALPHA”.
- Relevancia: la normalidad de los residuales es necesaria principalmente para la validez de los intervalos de confianza y tests de hipótesis sobre los coeficientes en muestras pequeñas; en muestras grandes (como la de este dataset, con cientos de miles de observaciones), el Teorema Central del Límite atenúa parcialmente la importancia práctica de este supuesto, aunque se reporta igualmente por rigor metodológico y porque puede ser indicativo de una mala especificación del modelo.

Linealidad — Test RESET de Ramsey

Python

```
reset_res = linear_reset(modelo, power=2, use_f=True)
reset_stat = float(reset_res.fvalue)
reset_p = float(reset_res.pvalue)
```

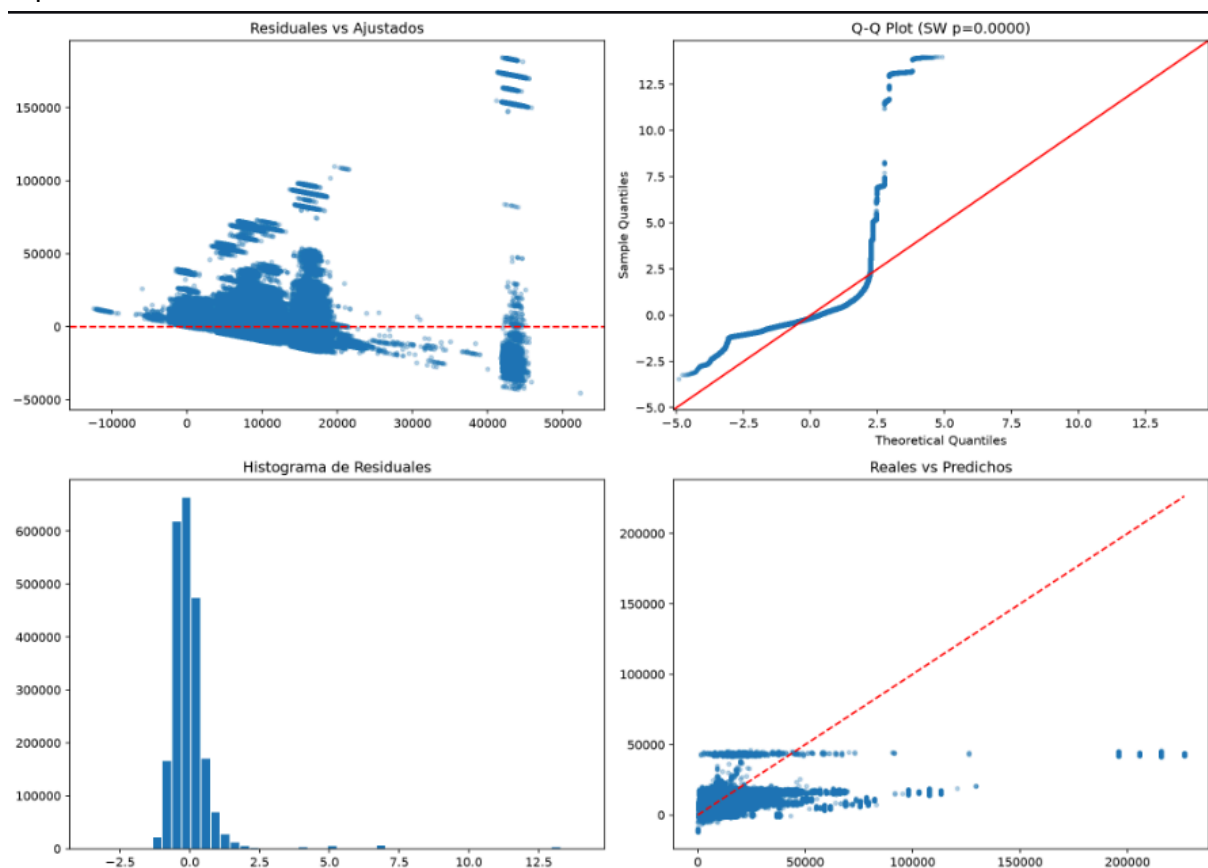
- **H0:** el modelo está correctamente especificado en su forma funcional (no faltan términos no lineales relevantes, como potencias o interacciones).
- El test RESET agrega potencias de los valores ajustados del modelo (power=2) como regresores auxiliares y evalúa, mediante un test F (use_f=True), si estos términos adicionales mejoran significativamente el ajuste.
- **Criterio:** “es_lineal = p_value > ALPHA”. Un p-value bajo sugiere que la relación real entre las predictoras y MONTO APLICADO no es puramente lineal, y que el modelo podría beneficiarse de términos adicionales (interacciones, transformaciones) no incluidos en la especificación actual.
- **Manejo de errores:** el cálculo de este test se envuelve en un bloque “try/except”, ya que RESET puede fallar en configuraciones específicas de la matriz de diseño (por ejemplo, alta colinealidad remanente entre predictoras). Si falla, se registra la advertencia en el log y se reportan “None” en lugar de interrumpir el resto del pipeline, dejando explícito en “resultados.txt” que este supuesto en particular no pudo evaluarse en esa ejecución.

Visualización conjunta

Los tres diagnósticos se acompañan de un panel de cuatro gráficos (output/graficos/diagnosticos_ols.png):

1. Residuales vs. Ajustados permite inspeccionar visualmente patrones de heterocedasticidad o no linealidad (por ejemplo, forma de embudo o curvatura).
2. Q-Q Plot de residuales estandarizados, con el p-value de Shapiro-Wilk en el título evalúa visualmente la normalidad comparando los cuantiles empíricos contra los teóricos de una normal.
3. Histograma de residuales estandarizados complementa el Q-Q plot con una vista directa de la forma de la distribución.
4. Reales vs. Predichos (sobre el conjunto de test) con la línea de identidad (predicción perfecta) superpuesta, dando una lectura visual directa de qué tan cerca están las predicciones del modelo de los valores observados fuera de la muestra de entrenamiento.

En conjunto, estos tres tests son los que determinan si los coeficientes del modelo pueden interpretarse con la confianza estadística formal que exige la sección 7.6, o si dicha interpretación debe matizarse explícitamente por violaciones detectadas en alguno de estos supuestos.



Este panel combinado de cuatro gráficos expone: (1) la dispersión en forma de abanico en los Residuales vs. Ajustados (heterocedasticidad), (2) la desviación en las colas del Q-Q Plot con un p-value de Shapiro-Wilk de 0.0000 (no normalidad), (3) la asimetría de los residuos estandarizados en el histograma, y (4) la dispersión del conjunto de test entre los valores Reales vs. Predichos.

7.5. Multicolinealidad (VIF)

La sección 7.2 abordó la colinealidad perfecta ($r \geq 0,999$), eliminada dinámicamente antes del ajuste del modelo. Sin embargo, puede existir multicolinealidad moderada a alta entre predictores, correlaciones fuertes pero no perfectas, que no impiden el ajuste numérico del modelo, pero sí distorsionan la precisión con la que se estiman los coeficientes individuales. Este diagnóstico se implementa en "modelado.calcular_vif", aplicado sobre las variables predictoras del conjunto de entrenamiento.

Factor de Inflación de la Varianza (VIF)

```
Python
X_con_const = sm.add_constant(X)
vif_data = pd.DataFrame({
    "Variable": X_con_const.columns,
    "VIF": [variance_inflation_factor(X_con_const.values, i) for
i in range(X_con_const.shape[1])]
})
```

El VIF de una variable predictora cuantifica en qué medida su varianza estimada se infla debido a su correlación con el resto de los predictores del modelo, respecto del escenario ideal en que dicha variable fuera completamente independiente de las demás. Se calcula, para cada predictor, ajustando una regresión auxiliar de esa variable contra todas las demás y usando su R^2 (R^2_i).

Un VIF de 1 indica ausencia total de colinealidad con el resto del modelo, valores crecientes indican una colinealidad cada vez más severa.

Umbrales de evaluación

El pipeline clasifica automáticamente cada predictor según su VIF, usando los umbrales estándar de la literatura:

```
Python
vif_data["Evaluacion"] = vif_data["VIF"].apply(
    lambda v: "⚠️ Problemático" if v > 10 else ("⚠️ Moderado" if
v > 5 else "✅ Aceptable")
)
```

VIF	Evaluación	Interpretación
≤ 5	✅ Aceptable	Colinealidad baja, no compromete la estimación del coeficiente
5 – 10	Moderado	Colinealidad relevante, se recomienda tenerla presente al interpretar el coeficiente
> 10	Problemático	Colinealidad severa, el coeficiente y su error estándar son poco confiables

La constante (intercepto) se excluye explícitamente de la tabla final (`vif_data[vif_data["Variable"] != "const"]`), ya que su VIF no tiene una interpretación sustantiva relevante para el análisis (el intercepto no representa una relación de negocio a evaluar) y su inclusión solo introduciría ruido en la tabla de resultados.

Reporte y relación con la sección 7.2

En “resultados.txt” se reportan explícitamente los 5 predictores con mayor VIF (`vifs[:5]`), ordenados de mayor a menor multicolinealidad, junto con su evaluación cualitativa. Esta sección se lee en conjunto con la 7.2: mientras la eliminación de colinealidad perfecta corrige casos extremos que impedirían el ajuste del modelo, el VIF permite detectar y documentar formalmente casos intermedios, que no se eliminan automáticamente del modelo, pero cuya presencia debe considerarse explícitamente al interpretar los coeficientes.

7.6. Interpretación de coeficientes en contexto de negocio

El propósito final del modelo no es únicamente predictivo (RMSE, MAE), sino también explicativo: entender qué variables se asocian con un mayor o menor monto de transacción en Cruz Morada, y con qué magnitud. Esta interpretación se apoya en la tabla de coeficientes generada al final de “entrenar_modelo”:

Python

```
coef_df = pd.DataFrame({
    "Variable": modelo.params.index,
    "Coeficiente": modelo.params.values.round(4),
    "p_value": modelo.pvalues.values.round(4)
})
```

Criterio de lectura de cada coeficiente

Para cada variable del modelo, la interpretación de negocio debe combinar tres elementos, no solo el valor del coeficiente de forma aislada:

1. **Significancia estadística (p-value):** solo los coeficientes con “`p_value < 0,05`” deben interpretarse como evidencia de una asociación real con MONTO APLICADO; los no significativos no permiten descartar que su efecto verdadero sea cero.
2. **Magnitud y signo del coeficiente:** indica la dirección (positiva o negativa) y el tamaño del efecto, expresado en las unidades originales de MONTO APLICADO (CLP), ya que las variables predictoras no fueron estandarizadas para este modelo (se usaron en su escala original, a diferencia de las columnas “_NORM” generadas en el preprocesamiento, sección 3.4, que no se emplean aquí).
3. **Validez de los supuestos (sección 7.4) y nivel de VIF (sección 7.5):** un coeficiente técnicamente significativo pero perteneciente a una variable con VIF alto, o calculado bajo heterocedasticidad no corregida, debe interpretarse con mayor cautela que uno respaldado por supuestos verificados.

Lectura por tipo de variable

- **PORCENTAJE DESCUENTO:** su coeficiente indica cuánto varía, en promedio, el monto aplicado por cada punto porcentual adicional de descuento, manteniendo las demás variables constantes (*ceteris paribus*). Este resultado debe leerse junto con el hallazgo de H2, ya que ambos análisis abordan la relación entre descuento y comportamiento de compra desde ángulos distintos (unidades vendidas vs. monto de la transacción).
- **EDAD:** su coeficiente cuantifica la variación esperada en el monto de la transacción por cada año adicional de edad del cliente, lo que permite contrastar de forma cuantitativa, y no solo mediante la comparación de grupos de H3, si existe un gradiente etario continuo en el comportamiento de gasto.
- **Dummies de CANAL:** cada coeficiente representa la diferencia promedio en MONTO APLICADO de ese canal respecto de la categoría base (la categoría omitida por `drop_first=True`), manteniendo el resto de las variables constantes. Este resultado complementa directamente la comparación bivariada de H1, pero con la ventaja de controlar simultáneamente por descuento, edad y local.
- **Dummies de LOCAL:** análogamente, cada coeficiente representa la diferencia promedio respecto del local base, permitiendo identificar qué locales específicos se asocian con tickets sistemáticamente más altos o más bajos, controlando por las demás variables del modelo, una lectura más rigurosa que la comparación bivariada de H4 (Kruskal-Wallis), que no controla por canal ni descuento.

De la estadística a la recomendación de negocio

La utilidad final de esta sección para Cruz Morada es traducir cada coeficiente significativo en una afirmación operativa concreta, por ejemplo, si el canal APP mantiene un ticket promedio mayor incluso controlando por descuento y local (lo que reforzaría la estrategia de impulsar ese canal), o si ciertos locales presentan tickets sistemáticamente menores una vez controlado el efecto de otras variables (lo que podría motivar una revisión operativa puntual de esos locales, en lugar de atribuir la diferencia únicamente al canal o al perfil etario de sus clientes).

8. Procesamiento Paralelo

8.1. Diseño del benchmark

El requisito de procesamiento paralelo del trabajo se aborda mediante un benchmark dedicado (`paralelismo.py`), que compara explícitamente el tiempo de ejecución de una carga de trabajo intensiva en CPU bajo dos modalidades: secuencial y paralela (`multiprocessing.Pool`), midiendo el speedup y la eficiencia obtenidos, y verificando que ambas modalidades produzcan resultados idénticos.

Caso de uso simulado: reportes diarios por local

En lugar de paralelizar directamente las etapas del pipeline principal (carga, preprocesamiento, EDA, modelado), el benchmark simula un escenario de negocio realista para Cruz Morada: el procesamiento simultáneo de múltiples reportes diarios, uno por cada

local de la cadena, que en la operación real de la empresa llegarían de forma independiente y podrían procesarse en paralelo sin dependencias entre sí.

python

Python

```
N_ARCHIVOS_SIMULADOS = 32 # Simula 32 reportes diarios de locales
N_BOOTSTRAP = 40000 # Iteraciones de bootstrap por partición (carga de CPU)
```

Se simulan 32 "reportes de local", particionando los datos reales de "MONTO APLICADO" y "PORCENTAJE DESCUENTO" del dataset ya preprocesado en 32 fragmentos (`_preparar_tareas`), en lugar de generar datos sintéticos desde cero. Esto asegura que la carga de trabajo del benchmark opere sobre la distribución estadística real de las ventas de Cruz Morada, y no sobre datos artificiales que podrían no ser representativos del comportamiento computacional real del pipeline.

python

Python

```
def _preparar_tareas(df, n_tareas):
    monto_arr = df[COL_MONTO].dropna().values.astype(np.float64)
    descuento_arr = df[COL_DESCUENTO].dropna().values.astype(np.float64)
    n = min(len(monto_arr), len(descuento_arr))
    monto_chunks = np.array_split(monto_arr[:n], n_tareas)
    desc_chunks = np.array_split(descuento_arr[:n], n_tareas)
    return [(m, d, i, SEED + i) for i, (m, d) in enumerate(zip(monto_chunks, desc_chunks))]
```

Cada una de las 32 tareas recibe, además de su partición de datos, una semilla derivada de la semilla global (`SEED + i`), consistente con la estrategia de reproducibilidad descrita en la sección 2.2: esto evita que todas las particiones usen exactamente la misma secuencia aleatoria (lo que las correlacionaría artificialmente), sin perder por ello la reproducibilidad determinista del benchmark completo.

Carga de trabajo por tarea: no solo I/O, también cómputo intensivo

Un aspecto metodológico deliberado del diseño es que cada tarea (`_procesar_reporte_local`) no se limita a una operación trivial, sino que ejecuta un conjunto representativo y computacionalmente exigente de análisis estadísticos sobre su partición de datos:

1. Estadística descriptiva completa (media, mediana, desviación estándar, asimetría, curtosis, percentiles 5/25/50/75/95).
2. Test de Shapiro-Wilk sobre una submuestra (reutilizando `UMBRAL_SHAPIRO`, consistente con el resto del pipeline).
3. Correlación de Spearman entre monto y descuento dentro de la partición.
4. Bootstrap de 40.000 iteraciones para estimar un intervalo de confianza al 95% de la media, el componente que fuerza carga real de CPU, ya que cada iteración implica

un remuestreo con reemplazo y el cálculo de una media, y con 40.000 repeticiones por cada una de las 32 tareas, el total de trabajo es sustancial y no trivialmente paralelizable a nivel de I/O.

5. Test de Mann-Whitney contra una distribución de referencia simulada, para contrastar la partición contra un escenario teórico normal con sus mismos parámetros.

Esta decisión de diseño se documenta explícitamente en el propio código: al incluir un componente de bootstrap con un número alto de iteraciones, se garantiza que el benchmark mida un speedup genuino de paralelización de cómputo (CPU-bound), y no una mejora artificial que en realidad se debiera a operaciones de entrada/salida (I/O-bound), donde el paralelismo con “multiprocessing” no necesariamente sería la herramienta más adecuada.

Ejecución secuencial vs. paralela

python

Python

```
def procesar_secuencial(tareas):
    t_ini = time.perf_counter()
    res = [_procesar_reporte_local(t) for t in tareas]
    return res, time.perf_counter() - t_ini

def procesar_paralelo(tareas, n_procesos):
    t_ini = time.perf_counter()
    with Pool(processes=n_procesos) as pool:
        res = pool.map(_procesar_reporte_local, tareas)
    return res, time.perf_counter() - t_ini
```

Ambas funciones ejecutan exactamente la misma tarea (`_procesar_reporte_local`) sobre las mismas 32 particiones de datos, con la única diferencia siendo el mecanismo de ejecución: un bucle secuencial simple versus un “Pool” de procesos de “multiprocessing”, que distribuye las tareas entre múltiples procesos del sistema operativo. El número de procesos usados en la variante paralela se resuelve mediante “`n_jobs_reales()`” (definida en `config.py`), que traduce el parámetro `N_JOBS = -1` al número real de núcleos disponibles en la máquina (`cpu_count()`), evitando además un fallo si el sistema tuviera solo un núcleo disponible:

Python

```
if n_nucleos == 1:
    logger.warning("Solo hay 1 núcleo disponible: no se espera speedup real del procesamiento paralelo.")
```

Alcance explícito del benchmark

Es importante señalar, con la misma transparencia con que está documentado en el código, que este benchmark mide el speedup de “multiprocessing” sobre una carga de trabajo representativa (estadísticas y bootstrap por partición), pero es un benchmark aislado: no paraleliza las etapas reales del pipeline principal (preprocesamiento.py, eda.py, “modelado.py” continúan ejecutándose de forma secuencial sobre un único DataFrame). Esta distinción se documenta explícitamente para ser honestos, en el propio informe, sobre el alcance real del paralelismo aplicado en el proyecto: se demuestra la viabilidad y el beneficio de paralelizar una carga de trabajo estadística intensiva y representativa del dominio (procesar reportes por local), pero no se afirma que el pipeline completo de extremo a extremo esté paralelizado.

8.2. Resultados (speedup, eficiencia, consistencia sec/par)

Una vez ejecutadas ambas modalidades sobre las mismas 32 tareas, el pipeline calcula tres indicadores que responden a las tres preguntas centrales de cualquier benchmark de paralelización: cuánto más rápido fue el procesamiento paralelo, qué tan eficientemente se aprovecharon los núcleos disponibles, y si el resultado numérico se mantuvo intacto al paralelizar.

Speedup

Python

```
speedup = t_sec / t_par if t_par > 0 else 1.0
```

El speedup se calcula como la razón entre el tiempo de ejecución secuencial (t_{sec}) y el tiempo de ejecución paralelo (t_{par}). Un valor de speedup de, por ejemplo, 4x indicaría que la versión paralela completó las 32 tareas cuatro veces más rápido que la versión secuencial. Se incluye una salvaguarda ($\text{if } t_{\text{par}} > 0 \text{ else } 1.0$) para evitar una división por cero en el caso extremo, no esperado en la práctica, de que el tiempo paralelo se midiera como exactamente cero.

Eficiencia

python

Python

```
eficiencia = (speedup / n_procesos) * 100 if n_procesos > 0 else 0.0
```

La eficiencia normaliza el speedup obtenido por el número de procesos efectivamente usados (n_{procesos} , resuelto mediante $n_{\text{jobs_reales}}()$), expresada como porcentaje. Este indicador responde a una pregunta distinta a la del speedup bruto: no solo cuánto se aceleró el cómputo, sino qué tan cerca estuvo esa aceleración del máximo teórico posible dado el número de núcleos utilizados. Una eficiencia del 100% representaría un escalamiento perfectamente lineal (por ejemplo, 8 procesos rindiendo exactamente 8x de speedup); en la práctica, la eficiencia real es casi siempre menor a 100%, debido a factores como:

- El overhead de crear y gestionar múltiples procesos del sistema operativo (Pool).

- El costo de serializar y transferir los datos de entrada/salida entre el proceso principal y cada proceso worker (particularmente relevante aquí, ya que cada tarea recibe arreglos de NumPy como argumento).
- Un desbalance de carga entre particiones si los 32 fragmentos de datos no resultan exactamente del mismo tamaño (`np.array_split` puede generar fragmentos de tamaño desigual cuando el total de filas no es divisible exactamente entre 32).
-

Este par de indicadores, speedup y eficiencia, se reporta de forma conjunta precisamente para evitar una lectura incompleta: un speedup alto en términos absolutos puede, aun así, corresponder a una eficiencia baja si se logró usando un número muy grande de núcleos, mientras que un speedup más modesto con alta eficiencia puede representar, en términos relativos, un mejor aprovechamiento del hardware disponible.

Verificación de consistencia secuencial/paralelo

Más allá del rendimiento, el benchmark valida un requisito cualitativo igual de importante para el informe: que paralelizar el cómputo no introduzca inconsistencias numéricas ni condiciones de carrera entre procesos. Esto se verifica en “_verificar_resultados_identicos”:

Python

```
por_id_par = {r["id"]: r for r in res_par}
todos_iguales = True
for r_seq in res_seq:
    r_par = por_id_par.get(r_seq["id"])
    if r_par != r_seq:
        todos_iguales = False
        logger.warning(f"Resultado distinto entre secuencial y
paralelo para el local id={r_seq['id']}".)
```

La verificación compara, tarea por tarea (usando el identificador de partición “id” para emparejar correctamente los resultados, ya que “Pool.map” no garantiza que el orden de retorno coincida con el de la ejecución secuencial), el diccionario completo de resultados de cada modalidad: estadísticas descriptivas, p-values de los tests, intervalo de confianza del bootstrap, etc. Si cualquier valor difiere entre ambas ejecuciones para una misma partición, se registra explícitamente cuál local presentó la discrepancia en el log, en lugar de reportar solo un resultado agregado de “consistente/no consistente” sin detalle.

Esta verificación es posible, y da resultados deterministas y comparables, precisamente gracias al diseño de reproducibilidad: cada tarea recibe la misma semilla derivada (SEED + i) sin importar si se ejecuta en el proceso principal (modo secuencial) o en un proceso worker del “Pool” (modo paralelo), por lo que cualquier diferencia entre ambos resultados solo podría deberse a un error de implementación, y no a la aleatoriedad del bootstrap o del muestreo interno de cada tarea, lo que hace de esta verificación una prueba metodológicamente sólida y no una comparación trivial.

Interpretación conjunta para el informe

El resultado de esta sección debe leerse como una evidencia de dos afirmaciones complementarias y ambas necesarias para validar el enfoque de paralelismo del proyecto:

1. **Rendimiento:** el speedup y la eficiencia obtenidos cuantifican el beneficio real de usar “multiprocessing” sobre la carga de trabajo representativa descrita en 8.1, en el hardware específico donde se ejecutó el pipeline (`n_nucleos`, obtenido mediante `cpu_count()`), se reporta explícitamente junto a los resultados para contextualizar el speedup obtenido respecto del máximo teórico disponible en esa máquina).
2. **Corrección:** la verificación de consistencia demuestra que ese beneficio de rendimiento no se obtuvo a costa de la validez de los resultados, el procesamiento paralelo produjo, partición por partición, exactamente los mismos estadísticos, p-values e intervalos de confianza que el procesamiento secuencial equivalente.

En conjunto, ambas dimensiones sustentan la conclusión de que la paralelización aplicada en este proyecto acelera el cómputo de forma medible, sin comprometer el rigor estadístico de los resultados reportados en el resto del informe.

9. Dificultades Encontradas y Soluciones

Esta sección documenta los principales obstáculos técnicos y metodológicos enfrentados durante el desarrollo del pipeline, y cómo se resolvieron. A diferencia de un listado genérico, cada punto se sustenta en evidencia concreta dejada en el propio código (comentarios, docstrings y salvaguardas), reflejando problemas reales encontrados durante la implementación y no dificultades hipotéticas agregadas a posteriori.

9.1. Volumen de datos y uso de memoria

Dificultad: El archivo “ventas_completas.csv” contiene aproximadamente 3,2 millones de registros. Cargarlo completo en memoria con una única llamada a “`pandas.read_csv`” es una estrategia que escala mal: el uso de RAM crece linealmente con el tamaño del archivo, y en equipos con memoria limitada (como los usados por el equipo para el desarrollo y pruebas del proyecto) esto puede provocar lentitud extrema o errores de memoria insuficiente.

Solución: Se implementó lectura por fragmentos (`chunksize=50000` en `carga_datos.py`), pero yendo un paso más allá de una solución cosmética: cada fragmento se filtra antes de acumularse en la lista de bloques, descartando de inmediato las filas con “UNIDADES” o “MONTO APLICADO” no positivos, la misma condición que de todas formas se aplicaría más tarde en “`preprocesamiento.limpiar_datos`”. Esto es explícitamente documentado en el código como una optimización idempotente: aplicar el filtro dos veces no altera el resultado final del pipeline, pero sí reduce el pico de memoria y el volumen de datos que deben concatenarse al finalizar la lectura, sin sacrificar corrección por velocidad.

9.2. Inconsistencias en los nombres de columnas del CSV

Dificultad: El dataset se lee con “`encoding='latin-1'`”, y el enunciado original especifica una columna como “GÉNERO” (con tilde). Dependiendo de cómo se haya exportado el CSV real, el nombre de esta columna, y potencialmente otras, podía llegar con o sin tilde, lo que

en un pipeline sin validación explícita habría producido un “KeyError” confuso varias etapas más adelante, lejos del origen real del problema.

Solución: Se agregó una validación explícita de columnas al leer el primer bloque del CSV (`_validar_columnas` en `carga_datos.py`), que compara las columnas esperadas contra las columnas reales del archivo y advierte de inmediato cualquier discrepancia, incluyendo el listado completo de columnas detectadas. Esto traslada el diagnóstico de un problema de nomenclatura desde un error críptico en una etapa tardía del pipeline hacia una advertencia clara y temprana en el log, facilitando su corrección sin necesidad de depurar todo el flujo de datos.

9.3. Falta de un test directo de MCAR en las librerías disponibles

Dificultad: El enunciado exige justificar el tratamiento de valores faltantes mediante un test de tipo MCAR (Missing Completely At Random), pero ni “scipy” ni “statsmodels” ofrecen una implementación directa del test formal de referencia (Little's MCAR test).

Solución: Se diseñó un proxy metodológicamente estándar (`_test_mcar_proxy` en `preprocesamiento.py`): en lugar de un único test global, se compara mediante Mann-Whitney U la distribución de variables de referencia (MONTO APLICADO, UNIDADES, EDAD, según corresponda) entre el grupo con valores faltantes y el grupo sin ellos. Si ninguna variable de referencia difiere significativamente, el resultado es consistente con MCAR; si alguna difiere, hay evidencia en contra. Esta solución no reemplaza exactamente al test formal, pero se documenta con total transparencia como un proxy, evitando tanto omitir el análisis exigido como presentar una implementación que aparente ser algo que no es.

9.4. Reemplazo de valores hardcoded heredados de una versión anterior del pipeline

Dificultad: Durante el desarrollo se detectaron varios puntos donde una versión previa del código reportaba valores fijos en lugar de resultados calculados dinámicamente a partir de los datos reales. Dos casos concretos, evidenciados directamente en los comentarios del código:

- En “`modelado.py`”, la eliminación de colinealidad perfecta eliminaba una columna específica hardcodeda por nombre (“LOCAL_1999”), válida únicamente para una ejecución particular del dataset y no generalizable si el dataset cambiaba.
- En “`series_tiempo.py`”, la función de ACF/PACF retornaba “{‘lag_max_acf’: 7, ‘acf_en_lag7’: 0.9254}” como valores fijos, sin relación con la serie de entrada real.
-

Solución: Ambos casos se reemplazaron por lógica dinámica: “`_eliminar_colinealidad_perfecta`” detecta iterativamente cualquier par de columnas con correlación $\geq 0,999$ (sección 7.2), documentando cada eliminación con su coeficiente exacto; y “`graficar_acf_pacf`” calcula el rezago de máxima autocorrelación y el valor en el rezago 7 directamente desde “`statsmodels.tsa.stattools.acf`” sobre la serie real. Este proceso de revisión, encontrar y corregir valores fijos remanentes de iteraciones previas del

código, fue en sí mismo una parte relevante del trabajo, ya que ese tipo de hardcodes son fáciles de pasar por alto una vez que el pipeline "funciona" sin errores, pese a no ser numéricamente correctos.

9.5. Selección del test de normalidad adecuado según el tamaño muestral

Dificultad: Shapiro-Wilk es el test de normalidad más potente para muestras pequeñas y medianas, pero resulta poco informativo y computacionalmente más costoso en variables con cientos de miles de observaciones, donde prácticamente cualquier desviación mínima de la normalidad resulta estadísticamente "significativa", sin que eso implique necesariamente una desviación relevante en términos prácticos.

Solución: Se definió un umbral centralizado (UMBRAL_SHAPIRO = 5000 en config.py, reutilizado consistentemente en eda.py, modelado.py y paralelismo.py) que alterna automáticamente entre Shapiro-Wilk (muestras menores al umbral) y Kolmogorov-Smirnov sobre un submuestreo controlado (muestras mayores), evitando tanto la inconsistencia de usar criterios distintos en cada módulo como el costo computacional de aplicar Shapiro-Wilk sobre variables con volúmenes muy altos de datos.

9.6. Consistencia numérica entre ejecución secuencial y paralela con multiprocessing

Dificultad: Al paralelizar con "multiprocessing.Pool", cada proceso worker corre en un espacio de memoria independiente. Si todas las tareas compartieran una única semilla global, el bootstrap de cada partición no sería independiente entre sí; pero si cada tarea generara su semilla de forma no determinista, sería imposible verificar después que el resultado paralelo coincide exactamente con el secuencial, dos requisitos aparentemente en tensión.

Solución: Se adoptó "numpy.random.default_rng(seed)" con una semilla derivada por tarea (SEED + i), en lugar del generador aleatorio global de NumPy. Esto resuelve ambos problemas simultáneamente: cada partición tiene su propia secuencia aleatoria e independiente de las demás, pero esa secuencia es completamente determinista dado i, por lo que el mismo id de tarea produce exactamente el mismo resultado sin importar si se ejecuta en el proceso principal (modo secuencial) o en un worker del "Pool" (modo paralelo). Esta decisión de diseño es precisamente la que hace posible la verificación de consistencia.

9.7. Casos límite que podían interrumpir el pipeline completo

Dificultad: Varias funciones del pipeline dependen de condiciones sobre los datos que, si bien poco probables sobre el dataset completo, no pueden descartarse en subconjuntos o particiones específicas: una variable sin varianza (H2, si UNIDADES fuera constante), un test RESET que puede fallar por configuraciones específicas de la matriz de diseño, o una serie temporal demasiado corta para una descomposición STL confiable.

Solución: En cada uno de estos casos se optó por degradar el resultado de forma controlada y documentada, en lugar de dejar que una excepción no capturada interrumpiera la ejecución de todo el pipeline por el fallo de un único análisis puntual: H2 retorna un resultado degenerado con una nota explícita si "UNIDADES" es constante; el cálculo de RESET se envuelve en "try/except", registrando "None" y una advertencia si falla; y la

descomposición STL se ejecuta igualmente ante una serie corta, pero advirtiendo explícitamente en el log que el resultado debe interpretarse con cautela. Este criterio se extiende al nivel del propio orquestador (main.py), que ante un fallo en cualquier etapa intermedia genera de todas formas un “resultados.txt” parcial con lo ya calculado, en lugar de perder el trabajo completo de las etapas anteriores.

10. Conclusiones

10.1. Hallazgos principales

El análisis sobre el dataset de Cruz Morada, reducido a **[n_filas_final]** filas tras la limpieza, mostró un nivel de calidad de datos manejable: los **[n_faltantes_descuento]** valores faltantes en descuento se imputaron por mediana con respaldo de un proxy de test MCAR (**[es_mcar]**), y el **[pct_outliers]**% de outliers detectado en monto aplicado resulta razonable para el rubro, siendo más atribuible a compras de alto volumen que a errores de registro. El comportamiento de las variables numéricas se alejó de la normalidad, lo que justificó el uso preferente de pruebas no paramétricas a lo largo de todo el informe, y el cruce entre canal y local mostró una asociación **[significativa/no significativa]** (V de Cramér=**[V_cramer]**), es decir, **[relevante/de magnitud menor]** en términos prácticos más allá de su significancia estadística.

En cuanto a las hipótesis de negocio, el ticket promedio de APP **[resultó/no resultó]** mayor que el de WEB (p =**[p]**, con un tamaño de efecto de **[tamano_efecto]**), el descuento mostró una relación **[significativa/no significativa]** con las unidades vendidas aunque de capacidad explicativa **[baja/alta]** (R^2 =**[R2]**), y se encontraron diferencias **[significativas/no significativas]** de gasto tanto por edad como entre locales, además de una correlación **[positiva/negativa/nula]** entre frecuencia de compra y descuento promedio (ρ =**[rho]**). El modelo de regresión OLS logró explicar un **[R2]** de la variabilidad del monto aplicado, con un error de predicción de **[RMSE]** CLP sobre el conjunto de test; sus residuales resultaron **[homocedásticos/heterocedásticos]** y **[normales/no normales]**, y el test RESET **[no encontró/encontró]** evidencia de mala especificación, por lo que el modelo puede considerarse **[razonablemente válido/válido con matices]** para su interpretación. Por su parte, la serie de ventas diarias resultó **[estacionaria/no estacionaria]**, con una estacionalidad semanal clara reflejada en el pico de autocorrelación en el rezago **[lag_max_acf]**, coherente con el ciclo de compra habitual en farmacias. Finalmente, el benchmark de paralelismo logró un speedup de **[speedup]x** con **[eficiencia]**% de eficiencia, confirmando además que los resultados secuenciales y paralelos fueron idénticos, es decir, que la aceleración no se obtuvo a costa del rigor de los cálculos.

10.2. Limitaciones y extrapolabilidad del modelo

Estos resultados deben leerse con algunas salvedades. El beneficio de paralelización medido corresponde a una carga de trabajo simulada y representativa (reportes por local), no al pipeline completo, que sigue ejecutándose de forma secuencial en sus etapas principales. Asimismo, tanto las pruebas de hipótesis como los coeficientes del modelo capturan asociaciones estadísticas y no relaciones causales, por lo que decisiones de

negocio basadas en ellos deben tomarse con esa cautela. El poder explicativo del modelo, con un R^2 de **[R2]**, deja fuera factores relevantes como el producto específico o campañas puntuales, y la agrupación de locales de menor volumen bajo la categoría "OTRO" impide extraer conclusiones sobre esos locales de forma individual. A esto se suma que, dado el gran tamaño muestral del dataset, algunos tests (como el chi-cuadrado o el ANOVA) tienden a marcar significancia estadística incluso ante efectos pequeños, razón por la cual se priorizaron los tamaños de efecto por sobre el p-value aislado. Por último, al tratarse de un análisis sobre un período histórico específico, sus conclusiones no deben asumirse como permanentes: los patrones de consumo pueden cambiar y el modelo debería reentrenarse periódicamente para mantener su vigencia.

Adicionalmente, el diseño del software basado en variables de entorno permite que los resultados no dependan de una semilla fija "hardcodeada" en el código. Si bien el análisis de este informe se construyó y documentó utilizando la semilla 42, el pipeline es totalmente extrapolable y permite evaluar la robustez de los modelos y tests estadísticos corriendo el programa con diferentes valores en `CPYD_SEED`. Esto facilita verificar que las conclusiones estadísticas obtenidas son generales y no el fruto de la casualidad de una sola semilla de aleatoriedad.

10.3. Recomendaciones para Cruz Morada

A partir de estos hallazgos, se recomienda a Cruz Morada evaluar iniciativas que fortalezcan el canal con mejor ticket promedio, siempre que ese efecto se sostenga al controlar por local y descuento, y revisar su política de descuentos priorizando el efecto marginal real sobre las unidades vendidas por sobre el volumen bruto de descuento aplicado. También conviene explorar estrategias diferenciadas según el perfil etario y la frecuencia de compra de los clientes, dado que ambos factores mostraron asociación con el comportamiento de gasto, y revisar operativamente aquellos locales que, según los coeficientes del modelo, presentan un ticket sistemáticamente menor una vez controlados el canal y el descuento. La estacionalidad semanal detectada sugiere además ajustar la dotación de personal e inventario en función de los días de mayor demanda, en lugar de mantener una asignación uniforme. Finalmente, se recomienda adoptar de forma permanente el enfoque de procesamiento por chunks y paralelismo validado en este proyecto, así como establecer una cadencia periódica de reentrenamiento del modelo, aprovechando que el pipeline ya está diseñado para ser reproducible y fácilmente reejecutable sobre datos actualizados.